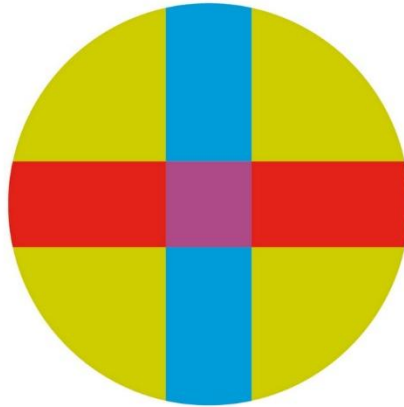


UNIVERSITY CEU - SAN PABLO

POLYTECHNIC SCHOOL

BIOMEDICAL ENGINEERING DEGREE



BACHELOR THESIS

Prompt Engineering for Collision Cross Section Prediction Using Locally Deployed Large Language Models

Author: Luis García de Juana

Supervisors: Guillermo Ramajo Fernández

June, 2026



UNIVERSIDAD SAN PABLO-CEU
ESCUELA POLITÉCNICA SUPERIOR
División de Ingeniería

Datos del alumno

NOMBRE:

Datos del Trabajo

TÍTULO DEL PROYECTO:

Tribunal calificador

PRESIDENTE:

FDO.:

SECRETARIO:

FDO.:

VOCAL:

FDO.:

Reunido este tribunal el ____/____/____, acuerda otorgar al Trabajo Fin de Grado
presentado por Don _____ la calificación de _____.

ACKNOWLEDGMENTS

Este trabajo marca el fin de una etapa muy importante de mi vida, que no habría sido posible sin las personas que me han acompañado en el camino.

En primer lugar, quiero agradecer a la Universidad CEU San Pablo, por la formación recibida, apoyo de los profesores y por darme la oportunidad de crecer, tanto a nivel académico como personal.

Quiero agradecer especialmente a mi tutor Guillermo, por su dedicación, orientación y acompañamiento de una manera tan cercana a lo largo de todo este trabajo.

Gracias a mis amigos, compañeros y en especial a mi novia, por todos los momentos compartidos, el apoyo recibido y por hacer de estos años una etapa inolvidable.

Gracias a mi familia, por ser un apoyo incondicional en todo momento, por su confianza y por ser el motor que me ha impulsado a llegar hasta aquí. En especial a mis padres, por todo su esfuerzo y sacrificio, y a mis abuelos que, aunque no han podido verme terminar, sé que estarían orgullosos de mí. Este trabajo también es vuestro.

Por último, darle las gracias a Dios por siempre guiarme, cuidarme y nunca abandonarme. Gracias por ser mi fortaleza y mi lugar seguro.

ABSTRACT

The identification of small organic molecules is a central challenge in analytical chemistry, with direct applications in metabolomics, drug discovery, and environmental monitoring. Ion mobility mass spectrometry addresses this by providing the Collision Cross Section as an additional structural descriptor that complements mass-to-charge ratio measurements and improves compound identification. As experimental CCS databases grow, predicting these values computationally has become increasingly valuable, reducing the number of candidate matches in database searches.

Existing prediction approaches range from physics-based methods to machine learning models, which are efficient but require task-specific training and produce predictions without interpretable justification. The rapid development of large language models has opened new possibilities for scientific prediction tasks, particularly with reasoning-oriented variants capable of integrating domain knowledge and producing structured justifications alongside their outputs.

This work demonstrates that a reasoning-oriented large language model, deployed locally on consumer hardware and guided by prompt engineering alone, can produce meaningful CCS predictions for small organic molecules without any task-specific fine-tuning. Four prompt variants were systematically evaluated on a balanced benchmark drawn from the METLIN-CCS dataset, quantifying the contribution of expert chemical knowledge and few-shot reference retrieval to prediction accuracy and exploring how these contributions scale with model size.

RESUMEN

La identificación de moléculas orgánicas pequeñas es un desafío central en química analítica, con aplicaciones directas en metabolómica, descubrimiento de fármacos y monitorización medioambiental. La espectrometría de masas con movilidad iónica aborda esto proporcionando la Sección Eficaz de Colisión como descriptor estructural adicional que complementa las mediciones de relación masa-carga y mejora la identificación de compuestos. A medida que crecen las bases de datos experimentales de CCS, predecir estos valores computacionalmente se ha vuelto cada vez más valioso, reduciendo el número de candidatos en las búsquedas.

Los enfoques de predicción existentes van desde métodos basados en física hasta modelos de aprendizaje automático, que son eficientes pero requieren entrenamiento específico para la tarea y producen predicciones sin justificación interpretable. El rápido desarrollo de los modelos de lenguaje de gran tamaño ha abierto nuevas posibilidades para tareas de predicción científica, especialmente con variantes orientadas al razonamiento capaces de integrar conocimiento de dominio y producir justificaciones estructuradas junto con sus respuestas.

Este trabajo demuestra que un modelo de lenguaje orientado al razonamiento, desplegado localmente en hardware de consumo y guiado únicamente mediante diseño de prompts, puede producir predicciones de CCS significativas para moléculas orgánicas pequeñas sin ningún ajuste específico para la tarea. Se evaluaron sistemáticamente cuatro variantes de prompt en un banco de pruebas equilibrado extraído del conjunto de datos METLIN-CCS, cuantificando la contribución del conocimiento químico experto y la recuperación de ejemplos por similitud a la precisión de las predicciones, y explorando cómo estas contribuciones escalan con el tamaño del modelo.

INDEX

1 INTRODUCTION	1
1.1 CONTEXT	1
1.1.1 The compound identification problem.....	1
1.1.2 Ion mobility mass spectrometry and the Collision Cross Section	1
1.1.3 Predicting CCS from molecular structure.....	2
1.1.4 Large language models as predictive tools.....	2
1.2 MOTIVATION.....	3
1.2.1 Limitations of existing CCS prediction approaches	3
1.2.2 The opportunity of reasoning-oriented LLMs	3
1.3 PROBLEM STATEMENT	3
1.4 OBJECTIVES	4
1.4.1 General objective	4
1.4.2 Specific objectives	4
1.5 SCOPE AND CONSTRAINTS.....	5
1.5.1 Chemical scope	5
1.5.2 Computational scope.....	5
1.5.3 Methodological scope	5
2 THEORETICAL FRAMEWORK.....	6
2.1 MOLECULAR DETERMINANTS OF THE COLLISION CROSS SECTION	6
2.1.1 Physical interpretation of CCS	6
2.1.2 Molecular factors influencing CCS.....	6
2.1.3 Experimental and computational determination.....	8
2.2 MOLECULAR REPRESENTATION: THE SMILES NOTATION	8
2.2.1 Structure and syntax of SMILES strings.....	8
2.2.2 Structural features extractable from SMILES	9
2.2.3 The InChIKey as a unique molecular identifier	9
2.3 ADDUCT TYPES IN ION MOBILITY MASS SPECTROMETRY.....	9
2.4 FOUNDATIONS OF LARGE LANGUAGE MODELS.....	11
2.4.1 Transformer architecture and autoregressive generation	11
2.4.2 Origin and training pipeline of the adopted model	11
2.4.3 Reasoning-oriented models and the DeepSeek-R1 family.....	12
2.4.4 Numerical precision and memory considerations	13
2.5 PROMPT ENGINEERING.....	13
2.5.1 The role of the prompt in LLM behavior.....	13
2.5.2 Few-shot prompting and similarity-based example retrieval.....	14

2.5.3 Chain-of-thought reasoning	15
2.5.4 Prompting guidelines for DeepSeek-R1	15
2.6 DECODING STRATEGIES IN AUTOREGRESSIVE LANGUAGE MODELS	16
2.6.1 Greedy and deterministic decoding	16
2.6.2 Sampling-based decoding	17
2.6.3 Repetition penalty	17
3 SYSTEM DEVELOPMENT	18
3.1 PROJECT SETUP AND ENVIRONMENT CONFIGURATION	18
3.1.1 Repository Structure and Directory Layout	18
3.1.2 Python Environment and Dependency Management	19
3.2 DATASET PREPARATION	20
3.2.1 Raw Data Exploration and Quality Assessment	20
3.2.2 Molecule-Level Train/Test Split Execution	20
3.2.3 Verification of Partition Integrity and Output Files	21
3.3 MODEL ACQUISITION AND LOCAL DEPLOYMENT	21
3.3.1 Automating Weight Downloads from Hugging Face Hub	21
3.3.2 Local Storage Structure and Integrity Verification	21
3.3.3 Enabling Fully Offline Operation	22
3.4 SUPPORTING INFRASTRUCTURE	22
3.4.1 Molecular Feature Extraction from SMILES	22
3.4.2 Adduct Normalization and Metadata	23
3.4.3 Similarity-Based Example Retrieval	23
3.4.4 Dataset Statistics	24
3.4.5 Experimental Setup: Fixed Test Bench	25
3.5 BUILDING THE CORE INFERENCE PIPELINE	25
3.5.1 Model and Tokenizer Loading	26
3.5.2 Quantization Strategies Evaluated	26
3.5.3 Generation Parameters	27
3.5.4 Response Extraction and Numeric Parsing	28
3.5.5 Prediction Classification	29
3.6 PROMPT ENGINEERING	30
3.6.1 Iteration 1 — Baseline Prompt: Molecular Descriptors Only	31
3.6.2 Iteration 2 — Adding Expert Chemical Rules	31
3.6.3 Iteration 3 — Adding Few-Shot Reference Examples	32
3.6.4 Iteration 4 — Full Combined Prompt with Heuristic Anchor	33
3.6.5 Final Prompt Structure	33
3.7 HARDWARE CONFIGURATION AND EXECUTION SETUP	33
3.8 WEB APPLICATION DEVELOPMENT	34

3.8.1 Backend Architecture and Initialization	35
3.8.2 REST API: /predict and /status Endpoints	35
3.8.3 Frontend interface.....	36
3.9 SYSTEM REFINEMENT AND FINAL INTEGRATION	37
3.9.1 Consolidation of Prompt and Parsing Logic into the Application	37
3.9.2 Interface Improvements and User Experience Refinements	37
3.9.3 End-to-End Validation on the Compound Test Bench	37
4 RESULTS.....	38
4.1 EVALUATION SETUP.....	38
4.2 QUANTITATIVE ACCURACY RESULTS	39
4.2.1 Results for the 1.5B model	39
4.2.2 Results for the 7B model	40
4.2.3 Cross-model comparison	40
4.3 PREDICTION QUALITY ANALYSIS	42
4.3.1 Distribution of prediction categories across prompt variants	42
4.3.2 Reference value copying as the primary failure mode	43
4.3.3 Mean anchoring across model sizes.....	43
4.3.4 Parsing fallback rate.....	43
4.4 WEB APPLICATION DEMONSTRATION	43
4.4.1 Initial state and query submission.....	43
4.4.2 Completed prediction.....	45
5 DISCUSSION.....	46
5.1 INTERPRETATION OF QUALITATIVE RESULTS	46
5.2 PROMPT ENGINEERING VS. MACHINE LEARNING	47
5.3 SYSTEM LIMITATIONS	47
5.4 FUTURE WORK.....	48
6 CONCLUSIONS.....	49
7 REFERENCES	50
8 APPENDIX: ARTIFICIAL INTELLIGENCE USAGE	53
8.1 WRITING, DRAFTING AND STYLE REVIEW	53
8.2 CODE GENERATION AND DEBUGGING	53
8.3 OTHER USES	53

FIGURE INDEX

FIGURE 1. CCS–M/Z TRENDS ACROSS CHEMICAL CLASSES. ADAPTED FROM ZHENG ET AL. [3].	7
FIGURE 2. TRANSFORMER ARCHITECTURE AND MULTI-HEAD ATTENTION MECHANISM.	11
FIGURE 3. STANDARD PROMPTING VS CHAIN-OF-THOUGHT PROMPTING [32].	15
FIGURE 4. PREDICTED VS. EXPERIMENTAL CCS VALUES FOR THE DEEPSEEK-R1-DISTILL-QWEN-1.5B AND 7B MODELS ACROSS THE FOUR PROMPT VARIANTS, EVALUATED ON THE 390-COMPOUND TEST BENCH. THE DASHED RED LINE REPRESENTS PERFECT PREDICTION ($X = Y$).	41
FIGURE 5. INITIAL STATE OF THE WEB INTERFACE.	44
FIGURE 6. WEB INTERFACE AFTER A SUCCESSFUL PREDICTION, SHOWING THE FOUR OUTPUT BLOCKS.	45

TABLE INDEX

TABLE 1. ADDUCT TYPES COVERED BY THE SYSTEM, WITH THEIR IONIC CHARGE, RELATIVE MASS CONTRIBUTION, AND EXPECTED EFFECT ON CCS.....	10
TABLE 2. PARSING CASCADE OF <i>PARSE_RESPONSE</i>	29
TABLE 3. CATEGORIES PRODUCED BY <i>CLASSIFY_PREDICTION</i>	30
TABLE 4. ACCURACY OF THE 1.5B MODEL ACROSS THE FOUR PROMPT VARIANTS, EVALUATED ON THE 390-COMPOUND TEST BENCH. THE BEST VALUE IN EACH COLUMN IS HIGHLIGHTED IN BOLD.....	39
TABLE 5. ACCURACY OF THE 7B MODEL ACROSS THE FOUR PROMPT VARIANTS, EVALUATED ON THE 390-COMPOUND TEST BENCH. THE BEST VALUE IN EACH COLUMN IS HIGHLIGHTED IN BOLD.....	40
TABLE 6. DISTRIBUTION OF PREDICTION CATEGORIES FOR THE 1.5B MODEL ACROSS THE FOUR PROMPT VARIANTS, EVALUATED ON THE 390-COMPOUND TEST BENCH.....	42
TABLE 7. DISTRIBUTION OF PREDICTION CATEGORIES FOR THE 7B MODEL ACROSS THE FOUR PROMPT VARIANTS, EVALUATED ON THE 390-COMPOUND TEST BENCH.....	42

1 INTRODUCTION

1.1 Context

1.1.1 The compound identification problem

The identification of small organic molecules is a central task in analytical chemistry, with direct applications in fields such as metabolomics, lipidomics, drug discovery, and environmental monitoring [1]. The dominant technique for this purpose is mass spectrometry, which characterizes compounds through their mass-to-charge ratio (m/z) with high sensitivity and precision [2]. However, the m/z value alone is often insufficient to clearly identify a molecule. Many compounds share the same nominal mass while differing in their three-dimensional structure, a situation that is especially common among isomers and structural analogues [3]. As a result, additional descriptors are required to separate candidate compounds, particularly in analytical workflows.

1.1.2 Ion mobility mass spectrometry and the Collision Cross Section

To address this limitation, ion mobility spectrometry has been coupled to mass spectrometry in an integrated technique known as ion mobility mass spectrometry (IM-MS). In IM-MS, ions are separated not only by m/z but also by their mobility as they travel through an inert buffer gas. This mobility is influenced by the ion's size and three-dimensional shape in the gas phase [4]. The property derived from this mobility measurement is the Collision Cross Section (CCS), an effective measure of the interaction area between the ion and the buffer gas, expressed in square ångströms ($\text{\AA}^2$) [5].

Because CCS depends on molecular geometry rather than on mass alone, it acts as an additional descriptor to m/z and provides discriminatory power between compounds that would otherwise appear identical in mass spectrometry [1]. The availability of predicted CCS values for candidate compounds is therefore highly valuable during compound identification workflows, as it allows experimental measurements to be compared against expected values and reduces the number of potential matches in a database search [6, 7].

1.1.3 Predicting CCS from molecular structure

However, predicting CCS from molecular structure is not simple. Physics-based approaches, such as the trajectory method or the projection approximation, provide rigorous estimates but require three-dimensional molecular models and involve computationally expensive simulations that scale poorly to large compound libraries [8]. Data-driven methods based on molecular descriptors and machine learning have emerged as practical alternatives, trained on the growing amount of experimental CCS data available in public repositories [9]. These methods are computationally efficient but typically require extensive feature engineering and generate predictions without an interpretable explanation of the underlying reasoning [10].

1.1.4 Large language models as predictive tools

In parallel with these developments, the rapid progress of Large Language Models (LLMs) has opened new possibilities for scientific prediction tasks. LLMs are deep neural networks trained on large text datasets that, once trained, can be adapted to new tasks simply by modifying their input rather than their parameters [11]. Recent reasoning-oriented variants are explicitly tuned to produce structured, multi-step reasoning before emitting a final answer, and can integrate domain knowledge expressed in natural language directly into the prompt [12].

These properties make LLMs an interesting candidate for physicochemical property prediction. Unlike conventional machine learning models, an LLM can combine numerical inputs, structural descriptors, expert rules, and reference examples within a single context, and its output can include an explicit justification of the predicted value [11]. The recent availability of distilled variants of such models, with significantly reduced memory requirements, enables their deployment on consumer-grade hardware, bringing this class of systems within reach of individual researchers and small laboratories [12].

1.2 Motivation

1.2.1 Limitations of existing CCS prediction approaches

Existing CCS prediction systems rely predominantly on classical machine learning or deep learning models that operate on engineered molecular descriptors or on learned molecular representations [10]. While such systems can achieve competitive accuracy, they share two practical limitations. First, they typically behave as black-box predictors that produce a numerical estimate without an associated explanation, which makes it difficult for the analyst to assess the reliability of the prediction in borderline cases [13]. Second, the integration of domain knowledge is implicit in the training data and cannot easily be modified or extended once the model is deployed [14].

1.2.2 The opportunity of reasoning-oriented LLMs

Reasoning-oriented LLMs offer a complementary approach that addresses both limitations. Domain knowledge can be incorporated explicitly through the prompt, in the form of natural-language rules, structural heuristics, or selected reference examples, and can therefore be modified without retraining the base model. The reasoning trace produced by the model, in turn, exposes the inference process to the user, allowing the predicted value to be inspected and contextualized. Despite this potential, the application of reasoning-oriented LLMs to CCS prediction remains comparatively unexplored, particularly under the limitation of running entirely on consumer hardware.

1.3 Problem Statement

The work presented in this thesis addresses the question of whether a reasoning-oriented LLM, deployed locally on consumer hardware and supported by a carefully designed prompting strategy, can produce informed and reproducible CCS estimates for organic molecules. The problem is limited by three practical considerations: the system must run without external inference APIs or specialized hardware, the predictions must include a structured justification to assess their plausibility, and the design must remain flexible to allow domain knowledge adjustments through the prompt rather than training.

1.4 Objectives

1.4.1 General objective

The general objective of this work is to design, implement, and evaluate a CCS prediction system for small organic molecules based on a locally deployed reasoning-oriented language model, supported by expert-informed prompt design and example retrieval from experimental data.

1.4.2 Specific objectives

The general objective is decomposed into a set of specific objectives that structure the work along its main stages. The first is to review the theoretical background required for the task, covering the physicochemical meaning of CCS, the role of the ionization adduct, the SMILES notation, and the relevant aspects of large language models, prompt engineering and decoding strategies. In parallel, a reference dataset of molecules is selected for implementation and testing, together with a reasoning-oriented language model family that can be deployed locally on consumer hardware, considering the trade-off between model size and memory requirements.

On this basis, the second and main objective is to design and systematically compare prompt engineering strategies for the CCS prediction task, through four prompt variants of increasing complexity evaluated on a balanced benchmark. This comparison aims to quantify the contribution of each prompting component and to identify the most effective configuration. The third objective is to implement the full inference pipeline, from user input to numerical prediction, including reference retrieval, prompt construction, model inference, and parsing of the reasoning traces produced by the DeepSeek-R1 family.

Finally, two further objectives address the deployment and evaluation of the system. The first is to assess the impact of model scale on prediction quality across the available variants of the model family, and the second is to expose the resulting system through a simple web interface that returns the predicted CCS together with a structured justification.

1.5 Scope and Constraints

1.5.1 Chemical scope

The system targets small organic molecules, consistent with the composition of the experimental dataset used in this work. Macromolecules fall outside the scope of the present system, as their CCS behavior and the available reference data are significantly different from those of small molecules. The set of ionization adducts considered is restricted to the three most common types present in the dataset, which together cover most experimental entries.

1.5.2 Computational scope

From a computational perspective, the system is designed to run locally on consumer hardware, without relying on external inference APIs or on specialized accelerators such as high-end GPUs. The main development environment corresponds to a laptop with 16 GB of system RAM and no dedicated GPU usable for inference. A secondary environment, a university workstation, is used exclusively for the comparative evaluation of larger model variants. These constraints directly shape the choice of the primary model and the loading configuration adopted, as detailed in Chapter 3.

1.5.3 Methodological scope

The predictive methodology is intentionally limited to inference-time techniques, using a pre-trained reasoning-oriented model in combination with prompt engineering and similarity-based retrieval of reference examples, while avoiding any fine-tuning or parameter updates. This choice is consistent with the limitations of the deployment environment and with the exploratory nature of the work, and it defines the boundary between what the system learns from data (the retrieval component) and what is handled by the pre-existing capabilities of the language model (the reasoning component). The evaluation focuses on the behavior and output quality of the system on the reserved test set, as well as on a qualitative comparison between the primary model and larger variants of the same family. A full comparison against other CCS prediction systems in the existing research is not part of the scope of this work and is left as a possible direction for future research.

2 Theoretical Framework

This chapter presents the theoretical foundations that support the design choices in Chapter 3. Building on the introductory material in Chapter 1, each section focuses on a specific component of the pipeline, ranging from the underlying chemical principles to the language model background that underpins the inference engine.

2.1 Molecular Determinants of the Collision Cross Section

2.1.1 Physical interpretation of CCS

From a physical standpoint, the Collision Cross Section (CCS) is the rotationally averaged effective area of interaction between an ion and the buffer gas as it drifts through an ion mobility cell [15]. Therefore, CCS is not a fixed geometric property of the molecule, but a dynamic quantity that depends on the conformations the ion can adopt in the gas phase, the nature of the buffer gas, and the experimental conditions of the measurement [16]. In public repositories, CCS values are reported together with the buffer gas used in the measurement (typically nitrogen) and follow standardized acquisition protocols, which makes them comparable across laboratories and suitable as targets for predictive modelling [17].

2.1.2 Molecular factors influencing CCS

The CCS of an ion is determined by a combination of intrinsic molecular properties, primarily size and mass, which establish a first-order trend where heavier ions tend to exhibit larger CCS values [15]. On top of this, three-dimensional shape plays a decisive role, as compact and rigid structures dominated by fused or polycyclic ring systems tend to produce smaller CCS values than flexible chains of similar mass, which can adopt more extended conformations in the gas phase [3]. Aromaticity acts as a secondary modulator by restricting the accessible conformations [15, 3], while polarizability further modulates CCS through its interaction with the buffer gas [18]. The ionization state and the associated adduct also affect charge distribution and geometry, but this factor is treated separately in Section 2.3.

Within this framework, more detailed structural effects modulate CCS for small organic compounds. Branched isomers fold into more compact conformations than their linear counterparts of equivalent mass, while bulky substituents increase the rotationally averaged area exposed to the buffer gas [3]. Heteroatoms (N, O, S) and halogens contribute to molecular mass but little to molecular volume, so their net effect on CCS is smaller than mass-based intuition would suggest [19]. Polar functional groups ($-\text{OH}$, $-\text{NH}_2$, $-\text{COOH}$) can also promote intramolecular interactions in the absence of solvent, slightly reducing CCS by stabilizing more compact conformations [15]. These trends are visible when plotting CCS against m/z , where lipids, peptides, oligosaccharides and small organic compounds occupy distinct regions of the space (Figure 1) [3].

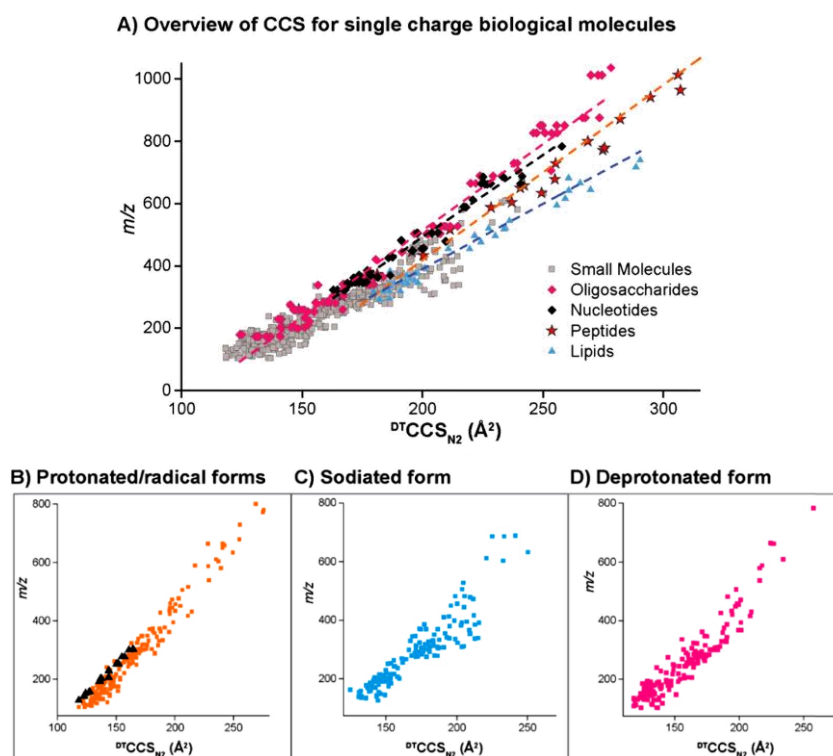


Figure 1. CCS- m/z trends across chemical classes. Adapted from Zheng et al. [3].

Not all these factors are equally accessible from the lightweight representations used in this work. Ring count and aromatic-atom count can be extracted directly from the SMILES string (Section 2.2.2), and molecular size can be approximated by its length. In the same order of importance, these three descriptors provide a simplified representation of the structural determinants of CCS and define the hierarchy of weights used by the similarity criteria in Section 3.4.3.

2.1.3 Experimental and computational determination

Experimentally, CCS values are obtained by calibrating drift times against reference standards of known cross-section on IM-MS instruments such as drift-tube or trapped ion mobility spectrometers [20]. Computationally, CCS can be estimated from three-dimensional molecular models using physics-based methods such as the trajectory method, the projection approximation, or the exact hard-sphere scattering method [21]. These approaches are precise but require explicit conformational sampling and are computationally expensive [8]. Data-driven approaches, including the one explored in this work, sidestep this cost by learning patterns from existing experimental data, at the price of relying on the representativeness of the training set [10].

For small organic molecules of the kind targeted in this work, experimentally measured CCS values typically fall in the range of 100-300 Å². This range serves as a qualitative reference for model outputs, while the response parser applies a broader acceptance window to account for edge cases, as described in Section 3.5.4.

2.2 Molecular Representation: the SMILES Notation

2.2.1 Structure and syntax of SMILES strings

The Simplified Molecular Input Line Entry System (SMILES) is a text-based notation for describing chemical structures as linear character strings [22]. It is compact and well-defined and has become one of the most widely used input formats for cheminformatics tools and for machine learning models applied to chemistry [22].

Atoms are written as their element symbols, bonds are represented by specific characters (such as =, # or :), branches are enclosed in parentheses, and ring closures are indicated by matching digits. Beyond these basic rules, SMILES also captures chemically meaningful details that distinguish otherwise equivalent structures. Lowercase letters denote aromatic atoms, and stereochemistry is encoded through additional symbols such as @ for stereocenters and / or \ for double-bond configuration [22].

2.2.2 Structural features extractable from SMILES

Although SMILES is a linear notation, several structural features relevant to CCS can be extracted directly from the string through simple parsing, without requiring a full chemical graph construction. Useful descriptors include the total length of the string as a proxy for molecular size, the number of ring-closure digits as an approximation of ring count, the number of opening parentheses as an indicator of branching, the count of lowercase atom symbols as a measure of aromaticity, and atom-symbol counts for specific heteroatoms (N, O, S, P), halogens (F, Cl, Br), stereocenters and net charge.

These lightweight descriptors do not replace full molecular descriptors (rings, length and aromatic atoms) derived from cheminformatics libraries [22], but they are computationally inexpensive and sufficient to capture first-order structural trends. In this project, a reduced subset of three descriptors serves as input to the similarity-based retrieval of reference examples described in Section 3.4.3, while the full set is exposed in the user interface for qualitative inspection.

2.2.3 The InChIKey as a unique molecular identifier

The International Chemical Identifier Key (InChIKey) is a fixed-length hashed representation derived from the full InChI string of a molecule [23]. Unlike SMILES, which can admit multiple equivalent representations for the same molecule, the InChIKey is canonical and therefore acts as a stable identifier for chemical entities. In this work, the InChIKey is used to ensure that the training and test sets do not share molecules, preventing data leakage and enabling a fair evaluation of generalization (Section 3.2.2).

2.3 Adduct Types in Ion Mobility Mass Spectrometry

In ion mobility mass spectrometry, molecules are rarely detected in their neutral form. Instead, during ionization they form charged species by either gaining or losing a small ion, and the resulting complex (the adduct) is what is ultimately measured. The identity of the adduct directly influences two observable quantities: the mass-to-charge ratio (m/z) recorded by the mass spectrometer and the collision cross section (CCS) measured in the ion mobility cell. Understanding adduct behavior is therefore essential for interpreting CCS values correctly, and it plays a central role in the prompt design

described in Section 3.4.2, where adduct-specific information is injected into the model's context to help it reason about the expected CCS.

The effect of an adduct on CCS can be understood in terms of two factors [26]. The first is the mass contribution of the adduct-forming ion, which slightly increases the overall size of the species detected relative to the neutral molecule. The second is the conformational effect of the ion. Larger counter-ions such as Na^+ or K^+ tend to produce more extended geometries than a simple proton, yielding slightly larger CCS values, whereas multiply charged species fold into more compact shapes due to intramolecular charge repulsion. Deprotonated species measured in negative mode typically produce slightly smaller CCS values than their protonated counterparts, as the absence of the proton leads to a marginally tighter ion shape [21].

Table 1 lists the three adduct types present in the METLIN-CCS dataset, together with their ionic charge, mass contribution relative to the neutral molecule, and expected effect on CCS. These correspond to the dominant ionization modes in positive and negative ionization modes and account for all experimental entries in the dataset. Atomic masses follow the IUPAC standard values [24], and the qualitative descriptions are adapted from the ion mobility mass spectrometry literature [21]. Support for additional adduct types is left as a direction for future work. These values are used to populate the Adduct Information Dictionary (*ADDUCT_INFO*) introduced in Section 3.4.2, which stores this information to use later during prompt construction.

Table 1. Adduct types covered by the system, with their ionic charge, relative mass contribution, and expected effect on CCS.

Adduct	Charge	Mass contribution	Effect on CCS
$[\text{M}+\text{H}]^+$	+1	+1.007	Standard reference, protonated form
$[\text{M}+\text{Na}]^+$	+1	+22.989	Sodium adduct, slightly larger CCS than $[\text{M}+\text{H}]^+$
$[\text{M}-\text{H}]^-$	-1	-1.007	Deprotonated, negative mode, typically smaller CCS

2.4 Foundations of Large Language Models

2.4.1 Transformer architecture and autoregressive generation

Modern LLMs are almost exclusively based on the transformer architecture (Figure 2) [25], whose core component is the self-attention mechanism. Self-attention allows the model to weigh the influence of every token in the input sequence when computing the representation of each position. While the original transformer was proposed as an encoder-decoder architecture for translation, modern generative LLMs such as the one adopted in this work use only the decoder stack.

During generation, LLMs operate in an autoregressive manner. At each step, the model receives the previously generated tokens and outputs a probability distribution over the vocabulary for the next token. A decoding strategy then selects a specific token from that distribution, which is appended to the sequence and fed back as input for the following step [26]. This process is repeated until a stopping condition is met and the decoding strategy used affects the behavior of the model, as discussed in Section 2.6.

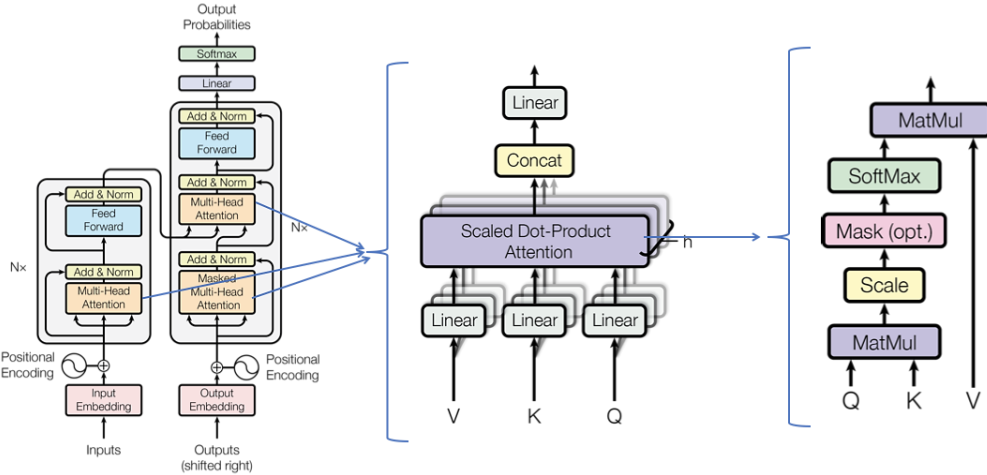


Figure 2. Transformer architecture and multi-head attention mechanism.
Adapted from Vaswani et al. [25].

2.4.2 Origin and training pipeline of the adopted model

The model used in this work is the result of a three-stage training pipeline applied by DeepSeek-AI [12]. It is loaded as a pre-trained checkpoint and used without any further parameter updates. Understanding how it was produced is nevertheless useful, since each of the three stages contributes to a specific capability that the system relies on.

In the pre-training phase, the model is trained on a large collection of texts to predict the next token in a sequence [26]. This is what gives it broad linguistic competence and exposure to chemical content, including the SMILES syntax used in the prompts of this work. A subsequent fine-tuning phase shapes the model's behavior using techniques such as supervised fine-tuning and reinforcement learning from human feedback [27], and is what enables it to follow the structured output instructions embedded in the prompt (Section 3.6). Finally, knowledge distillation transfers the capabilities of a larger teacher model to a smaller student model [28]. The DeepSeek-R1-Distill-Qwen variants used in this project are the result of this process, which makes them small enough to run on the consumer-grade hardware available (Section 3.3).

2.4.3 Reasoning-oriented models and the DeepSeek-R1 family

Within the broader landscape of LLMs, a distinction can be drawn between models optimized for fluent conversational generation and those tuned for structured multi-step reasoning [12]. The DeepSeek-R1 family belongs to the latter category, designed for tasks that benefit from explicit intermediate reasoning such as mathematical problem solving and scientific inference [12]. This category is the natural fit for CCS prediction, which requires applying a sequence of structural and chemical considerations before committing to a numerical answer.

Among the reasoning-oriented model families available, DeepSeek-R1 was selected based on three criteria. First, the weights are openly released [12], enabling fully local deployment without recurring API costs or external service dependencies. Second, the family is published together with distilled variants of substantially smaller size, essential for execution on the consumer-grade hardware available (Section 3.3). Third, R1 models produce long reasoning traces delimited by explicit `<think>` and `</think>` markers before emitting a final answer [12], a behavior exploited directly by the prompt construction (Section 3.6) and the response parser (Section 3.5.4). Within the distilled set, the Qwen-based variants were selected over the Llama-based alternatives, since the Qwen line retains more of the teacher's reasoning capability at this scale [12]. The 1.5B and 7B checkpoints provide the two model sizes for the comparative evaluation in Chapter 4.

2.4.4 Numerical precision and memory considerations

The memory footprint of an LLM is determined primarily by the number of parameters and the numerical precision used to store its weights. PyTorch models use 32-bit floating point precision (float32) by default, although lower-precision formats such as float16 and bfloat16 reduce memory usage while maintaining enough numerical accuracy for inference [29]. Using float16 roughly halves the memory footprint compared with float32, which can determine whether the model fits within the available hardware.

Beyond the choice of precision, additional optimizations can be applied at loading time. Weights can be streamed directly to their target device to avoid CPU memory peaks during initialization. More aggressive techniques, such as quantization to 8-bit or 4-bit integer representations, offer greater memory savings at the cost of increased complexity and potential accuracy trade-offs. The loading configuration adopted in this project, together with the alternatives evaluated, is described in Sections 3.5.1 and 3.5.2.

2.5 Prompt Engineering

Because no fine-tuning is performed on the language model, the prompt is the only mechanism through which the task is defined, the domain knowledge is injected, and the output format is constrained. This section reviews five prompting principles that the system relies on, presented in the order in which they combine inside a single prompt and which together underpin the four prompt iterations evaluated in Section 3.6.

2.5.1 The role of the prompt in LLM behavior

The prompt is the textual input that conditions the generation of an LLM. Since modern LLMs are not fine-tuned for every new task and are instead adapted through their input, prompt design has a strong effect on the quality and consistency of the output [30]. Prompt engineering refers to the design of prompts to elicit the desired behavior from the model, typically by specifying the task, providing relevant context, and defining the output format. Even small variations in wording, ordering, or structure can have an effect on the model output. In scientific applications, prompts also serve as a vehicle for injecting domain knowledge that is not reliably present in the model's training data.

2.5.2 Few-shot prompting and similarity-based example retrieval

Few-shot prompting is a technique in which a small number of worked examples are included in the prompt, allowing the model to infer the task by analogy without any update to its parameters [11]. This exploits the in-context learning capability of LLMs, in which the model adapts its behavior based on the patterns present in the prompt. Its applicability to reasoning-oriented models is more nuanced than it is for general-purpose LLMs, a point addressed in Section 2.5.4.

The effectiveness of few-shot prompting depends not on the number of examples but on their relevance to the query [30]. When the set of available references is large, including all of them in the prompt is neither practical nor desirable, since language models have a finite context window. A selection mechanism is therefore required to retrieve a small subset of examples that are most informative for the specific question. This is the underlying principle of retrieval-augmented approaches to prompting [31], in which reference entries are dynamically selected from an external corpus at inference time and inserted into the prompt as context.

A simple and effective form of retrieval is similarity-based selection, in which candidate references are ranked according to a similarity measure defined between each candidate and the query, and the top-ranked entries are then included in the prompt. The choice of similarity measure depends on the nature of the data. For text-only tasks, similarity is typically computed in an embedding space produced by a separate encoder model [31]. For structured or domain-specific data, similarity can be defined directly over interpretable features extracted from the input, which is computationally cheap and transparent at the cost of relying on hand-designed descriptors rather than learned representations [30].

In this work, similarity is defined using structural descriptors derived from each compound's SMILES string, so that retrieved references are structurally similar to the query rather than only textually similar. The specific descriptors, weights, and filtering rules are described in Section 3.4.3.

2.5.3 Chain-of-thought reasoning

Chain-of-thought prompting is a technique that encourages the model to produce an explicit sequence of intermediate reasoning steps before emitting the final answer (Figure 3) [32]. Reasoning-oriented models such as DeepSeek-R1 are natively tuned to produce chain-of-thought traces (Section 2.4.3), and this behavior is leveraged in this work to encourage the model to reason about molecular structure and adduct effects before committing to a numerical CCS estimate.

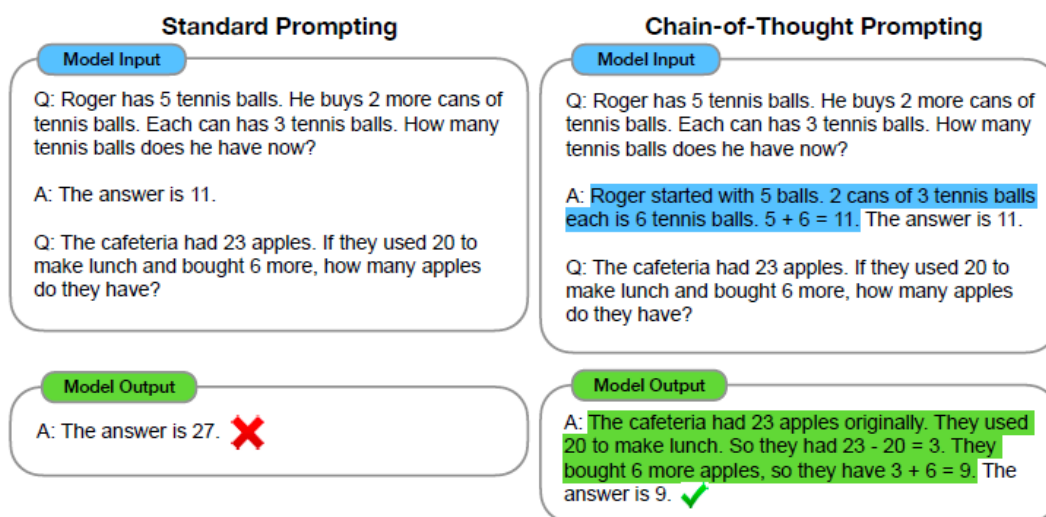


Figure 3. Standard prompting vs chain-of-thought prompting [32].

2.5.4 Prompting guidelines for DeepSeek-R1

The DeepSeek-AI team publishes a short set of recommendations for the R1 family in the official model card released alongside the model weights [33]. Four of them shape the prompt design adopted here, and one is followed only partially.

The first recommendation is to place all task instructions inside the user prompt and to avoid a separate system prompt, since the R1 training process tuned the model to respond to direct user queries [33]. This work follows the guidelines by concatenating the entire prompt into a single user-side string, without any system role.

The second recommendation concerns the language of the prompt. R1 models are optimized for English and Chinese, and the official paper documents language-mixing artefacts when the prompt is in a different language [12]. The prompts in this work are written entirely in English to remain within the well-supported regime.

The third recommendation is to use a sampling temperature between 0.5 and 0.7 and a top-p value of 0.95 [33]. These ranges introduce just enough variability to prevent the model from falling into the repetition loops that greedy decoding can cause. The exact values adopted for each model size are reported in Section 3.5.3.

The fourth recommendation is to explicitly open the reasoning block with a `<think>` tag at the end of the prompt, because the model occasionally bypasses the thinking pattern on its own [33]. All four prompt variants used in this work end with an opening `<think>` tag followed by a short reasoning stem (Section 3.6).

A further recommendation concerns few-shot prompting. While the technique generally improves performance in LLMs [11], R1 models tend to perform worse with in-context examples than without [12]. Two of the four prompt variants evaluated in Section 3.6 therefore include reference molecules with known CCS values as numerical anchors and the other two do not, so the actual impact of few-shot prompting on DeepSeek-R1-Distill-Qwen is measured experimentally.

2.6 Decoding Strategies in Autoregressive Language Models

The decoding strategy controls how each new token is selected from the probability distribution produced at every generation step, and has a direct effect on the determinism, fluency, and diversity of the output [34]. It is one of the few aspects of the inference pipeline that can be adjusted without retraining the model.

2.6.1 Greedy and deterministic decoding

Greedy decoding is the simplest decoding strategy, where the token with the highest predicted probability is selected at each step. This approach makes the process fully deterministic and easy to reproduce, making it suitable for tasks where consistency is prioritized over diversity. Greedy decoding is especially attractive for regression-like tasks with a single correct numerical answer, but it can favor repetitive loops in long reasoning traces, an effect more frequent in larger reasoning-oriented models [34]. The decoding mode adopted for each model variant is described in Section 3.5.3.

2.6.2 Sampling-based decoding

Sampling-based decoding introduces controlled randomness into token selection by drawing from the predicted probability distribution rather than always choosing the highest-probability token. This approach is useful when greedy decoding produces degenerate or repetitive outputs, particularly in long reasoning sequences. Sampling is enabled by setting `do_sample=True` in the Hugging Face transformers library and is controlled through two main parameters, temperature and top-p.

Temperature scales the logit distribution before sampling. A value of 1.0 leaves the distribution unchanged, values below 1.0 make the distribution sharper and the model more conservative, while values above 1.0 flatten it and increase diversity at the cost of coherence [34]. For reasoning-oriented tasks, low temperatures (typically below 1.0) are preferred to keep the reasoning trace focused while still introducing enough variability to escape repetitive loops [12, 33].

Top-p sampling, also known as nucleus sampling, restricts the candidate set at each step to the smallest group of tokens whose cumulative probability exceeds a threshold p , and samples only from this group [34]. Typical values of p lie between 0.9 and 0.95 [34]. Top-p sampling avoids selecting low-probability tokens that could derail the generation while still preserving sufficient diversity to escape repetitive patterns.

2.6.3 Repetition penalty

In addition to temperature and top-p, generation can be further controlled by applying a repetition penalty during decoding [35]. A value of 1.0 applies no penalty, while values above 1.0 progressively suppress repetition. In autoregressive reasoning models, mild penalties in the range 1.1-1.2 are typically used to discourage degenerate loops without distorting the natural fluency of the generated text [35]. The specific value adopted in this work, and its motivation in the context of DeepSeek-R1 distilled variants, are reported in Section 3.5.3.

3 SYSTEM DEVELOPMENT

This chapter describes the step-by-step implementation of the CCS prediction system, following the chronological order in which each component was developed. The objective is to provide a complete and detailed description of the construction process, from initial environment setup to the final integrated application, so that an independent reader can rebuild the system from scratch.

3.1 Project Setup and Environment Configuration

The first step in the implementation was establishing a reproducible development environment, structured as a standard Python repository in which each component of the pipeline could be developed and tested independently.

3.1.1 Repository Structure and Directory Layout

The repository follows a flat, modular layout organized around the data lifecycle. All datasets reside under a *data/* directory, which is further subdivided into three subdirectories: *raw/* for the original METLIN-CCS CSV file as obtained from the source, *processed/* for the molecule-level train/test partition produced by the preprocessing script, and *results/* for the timestamped output files generated by the prompt engineering experiments (one *.xlsx* prediction table and one stdout *.txt* log per run).

Experimentation and utility code is collected under *scripts/*. This directory contains the Python files used to download the model weights (*download_models.py*) and to inspect and partition the dataset (*explore_dataset.py*, *split_dataset.py*), together with the two main experimental procedures of the project. The first of these, *efficiency_tests.py*, was developed early in the project as an initial end-to-end prototype. It loads the model, applies a single baseline prompt to a fixed set of six compounds, and was used to converge on the appropriate values for tokenizer configuration, generation parameters, response parsing, and prediction classification. The second script, *prompt_tests.py*, was built on top of the resulting pipeline once it had stabilized. It reuses the same model loading and inference machinery and extends the evaluation to four prompt variants applied to an extended bench of 390 compounds, ensuring consistent saving of results to *.xlsx* and full execution logging to *.txt*.

The web frontend template is isolated under *templates/*, following the structured expected by Flask. The root of the repository contains the main application entry point (*app.py*), the dependency manifest (*requirements.txt*), and the runtime configuration file (*.env*). Pre-trained model weights are not stored inside the repository. Instead, they reside on an external hard drive that is connected to the workstation whenever the system is run, and the application reads them through a configurable base path (Section 3.3.2). This decoupling keeps the repository lightweight and allows the same model storage to be shared between the personal laptop and the university workstation (Section 3.7).

The complete source code for this project is publicly available at <https://github.com/luisgdj/FinalProject>, where all scripts, configuration files, and the web application template can be found. The repository also includes a *docs/* folder containing the final results spreadsheet and a PDF copy of this thesis.

3.1.2 Python Environment and Dependency Management

The project targets Python 3.11. Core runtime dependencies declared in *requirements.txt* fall into two groups according to the role they play in the system.

The first group covers model loading and inference. The Hugging Face transformers library provides the *AutoModelForCausalLM* and *AutoTokenizer* interfaces used to load each DeepSeek-R1-Distill-Qwen variant from its local directory, and *huggingface_hub* provides the *snapshot_download* helper used by the acquisition script (Section 3.3.1). The actual tensor computation is performed by PyTorch, configured to run on CPU only, in line with the hardware available in this project (Section 3.7).

The second group covers the application and experimentation layer. Flask serves as the web framework for the final user interface, exposing the prediction endpoints described in Section 3.8. The prompt engineering script (*prompt_tests.py*) uses *openpyxl* to persist per-compound predictions incrementally to *.xlsx* files. Each prediction is flushed to disk immediately after it is produced, so that no results are lost if a long-running experiment is interrupted before completion. Together with the timestamped *.txt* logs produced by the same script, these two outputs provide a complete audit trail of every prompt engineering run carried out during the project.

3.2 Dataset Preparation

Before any model work could begin, the reference dataset needed to be acquired, inspected, and partitioned into training and test subsets. This stage was completed early in the development process and its output remained fixed through all experimentation.

3.2.1 Raw Data Exploration and Quality Assessment

The reference dataset used in this work is METLIN-CCS [36], a collection of experimental CCS values for small organic molecules measured by ion mobility mass spectrometry, downloaded as a CSV file and placed in `data/raw/`. An initial exploration script (`explore_dataset.py`) confirmed the dataset dimensions and the presence of all required columns: `smiles`, `InChIKey`, `m/z`, `Adduct`, and `CCS_AVG`. The file contains 61,868 measurement rows, with CCS values ranging from 121.48 to 277.46 Å² and a mean of approximately 185 Å², covering the three adduct types described in Section 2.3.

3.2.2 Molecule-Level Train/Test Split Execution

Data preprocessing was carried out in `split_dataset.py`. To prevent data leakage between the training and test subsets, the split was performed at the molecule level rather than the row level. All unique molecules were first identified by their `InChIKey`, which serves as a canonical structural fingerprint independent of the adduct. These identifiers were then partitioned 80/20 into training and test sets by random sampling, with a fixed random seed to guarantee reproducibility across re-runs of the script. Dimer entries are excluded, as their CCS values do not correspond to the individual molecule described by the SMILES. The two resulting subsets were finally materialized by selecting from the cleaned dataset all rows whose `InChIKey` belonged to each side of the partition.

The training set (`train.csv`) is used exclusively as the pool from which similar reference molecules are retrieved to build the few-shot section of the prompt (Section 3.4.3), while the test set (`test.csv`) is reserved for evaluation. The resulting subsets contain:

- Training set (`train.csv`): 46,391 measurement rows, CCS range 121.48-252.02 Å²
- Test set (`test.csv`): 11,650 measurement rows, CCS range 122.27-249.03 Å²

3.2.3 Verification of Partition Integrity and Output Files

After the split, a verification step confirmed that no *InChIKey* appeared in both subsets simultaneously. Both CSV files were saved to *data/processed/* and loaded at application startup. This partition was fixed for the entirety of the project, ensuring that all evaluation results reflect genuine generalization to unseen molecules.

3.3 Model Acquisition and Local Deployment

To fully enable offline and reproducible operations, all model weights were downloaded from the Hugging Face Hub and stored locally before inference. The models used in this work belong to the DeepSeek-R1-Distill-Qwen family, a set of reasoning-oriented language models distilled from the larger DeepSeek-R1 checkpoint into smaller architectures based on Qwen. Two variants of this family were used in the experimental work of this project, the 1.5B and 7B parameter checkpoints.

3.3.1 Automating Weight Downloads from Hugging Face Hub

A dedicated script (*download_models.py*) was developed to handle model acquisition, following the usage pattern recommended in the Hugging Face model cards. The script is parameterized with the list of repository identifiers and relies on the *snapshot_download* function from the *huggingface_hub* library to retrieve every file of each repository into a configurable local directory. Additionally, the script checks if the files already exist before overwriting and can also resume interrupted downloads.

3.3.2 Local Storage Structure and Integrity Verification

Each model is stored in its own subdirectory under a configurable base path on the external hard drive introduced in Section 3.1.1. After every download, the script runs a verification step that checks for the presence of the files required to load the model with the Hugging Face *transformers* library: *config.json*, *tokenizer_config.json*, *tokenizer.json*, and at least one *.safetensors* weight shard. It also reports on *tokenizer.model*, a *SentencePiece* file not used by the Qwen-based models of this work, whose absence is therefore expected. The verification step ends with a summary that prints, for each model, the status of each file and the total disk size of the downloaded directory.

3.3.3 Enabling Fully Offline Operation

Once the weights are stored locally, every subsequent model operation references the local directory directly through a *model_path* variable, with no further calls to the Hugging Face Hub at inference time. This guarantees reproducibility regardless of network availability and removes runtime dependencies on remote model repositories.

3.4 Supporting Infrastructure

Before describing the core inference pipeline and the prompt engineering process, this section introduces a set of auxiliary components that form the shared foundation of the entire system. These components are used transparently by the experimental scripts (*efficiency_tests.py* and *prompt_tests.py*) and by the final web application (*app.py*), ensuring consistent behavior across development and deployment. Presenting them here avoids redundant explanations and highlights their role as general building blocks of the system.

3.4.1 Molecular Feature Extraction from SMILES

The function *extract_features* derives a compact set of structural descriptors directly from a SMILES string, without relying on any external cheminformatics library. This keeps the system lightweight and fully self-contained, at the cost of using surface-level character counts rather than a proper chemical parse.

The descriptors computed include the total length of the SMILES string, the number of rings (approximated by counting digit characters, which act as ring-closure labels in SMILES notation), the number of branches (opening parentheses), the counts of double and triple bonds (= and # characters), the number of aromatic atoms (lowercase c, n, o, s, p), the counts of specific heteroatoms (N, O, S, P) and halogens (F, Cl, Br), the number of stereocenters (@ symbols), and the net charge (difference between + and - symbols). These descriptors are later used both to populate the target block of the prompt and as input features for the similarity function described in Section 3.4.3.

3.4.2 Adduct Normalization and Metadata

In the source dataset, adduct labels are stored without their trailing charge sign, but a user querying the system may provide the same adduct including it (for example, writing $[M+H]^+$ instead of $[M+H]$). The helper function *normalize_adduct* handles this mismatch by stripping the trailing charge sign from any input adduct string, ensuring that the user's query and the stored dataset labels can be compared using the same format. The normalized form is the label used everywhere else in the pipeline when adducts need to be compared or looked up.

A static dictionary, *ADDUCT_INFO*, encodes the adduct metadata introduced in Section 2.3 (ionic charge, mass contribution, and effect on CCS) in a form the system can access programmatically. At prompt construction time, the normalized adduct of the query is used as a lookup key to retrieve the corresponding entry, and the resulting values are inserted directly into the target block of the prompt. The three adduct types observed in the dataset are fully covered by the dictionary. This design ensures that the model receives an explicit description of the ionization mode rather than having to infer it from the adduct label alone.

3.4.3 Similarity-Based Example Retrieval

The function *select_examples* selects, for a given molecule, the most structurally similar compounds from the training set. These selected compounds are later injected into the prompt as few-shot references, allowing the model to ground its numerical estimate in experimentally measured CCS values rather than reasoning purely from abstract rules. The procedure consists of three steps.

First, the training set is filtered to keep only compounds that share the same normalized adduct as the query, since CCS values depend strongly on the adduct type and comparing across different adducts would mix incompatible references. If this filter leaves fewer than the established number of references, the full training set is used instead.

Second, each surviving candidate is scored against the query through a weighted combination of three structural descriptors derived from the SMILES string (number of rings, length of SMILES, and number of aromatic atoms). Each descriptor is converted

into an individual similarity value between 0 and 1 using the transformation $\text{sim} = 1/(1 + |\Delta x|/\text{scale})$, which yields 1 when the two compounds share the same value of the descriptor and 0.5 when their difference equals the scale parameter. The three individual similarities are then combined into a single score by the following weighted sum:

$$\text{Similarity} = 0.50 \cdot \text{sim}_{\text{rings}} + 0.30 \cdot \text{sim}_{\text{length}} + 0.20 \cdot \text{sim}_{\text{aromatic}}$$

The weights of the three components reflect the relative importance of each descriptor for CCS, as established in the theoretical framework (Section 2.1.2). The specific weights and scale factors were not optimized systematically but chosen by hand to be consistent with this hierarchy of importance.

- The number of rings carries the most direct link to the three-dimensional shape of the ion and therefore receives the largest weight (0.50).
- The SMILES length acts as a proxy for overall molecule size and receives the next largest weight (0.30).
- The aromatic-atom count acts as a secondary indicator and receives the remaining weight (0.20).

Third, the candidates are sorted by their final similarity score in descending order and returned to be used as few-shot examples in the prompt.

3.4.4 Dataset Statistics

At application startup, the function *analyze_data* computes a small set of global statistics over the training set and keeps them in memory for the rest of the session. The statistics consist of three descriptors of the CCS distribution.

The minimum and maximum are embedded into the prompt to give the model an explicit plausible range for the predicted value, anchoring it to the region of CCS space actually observed in the data and discouraging numerically implausible outputs. The mean is used by the prediction classification step described in Section 3.5.5 to flag predictions that fall exactly on the dataset average, since such outputs typically indicate that the model has defaulted to a global statistic rather than producing a genuine estimate.

3.4.5 Experimental Setup: Fixed Test Bench

Two complementary evaluation sets, both drawn from `test.csv`, were established early in the project and kept fixed throughout all subsequent experiments. A small bench of six compounds (two per adduct) is used by the efficiency study (*efficiency_tests.py*), which is small enough to be evaluated repeatedly under different hardware and decoding configurations. An extended bench of 390 compounds is used by the prompt engineering study (*prompt_tests.py*), which provides a more statistically meaningful comparison between the four prompt variants while remaining tractable on CPU-only hardware. The six entries of the efficiency bench correspond to the first two entries of each adduct block in the extended bench, so every compound evaluated in the efficiency study is also evaluated in the prompt engineering study under identical pipeline conditions.

The 390 compounds of the extended bench were selected to balance the three adduct types present in the dataset, with exactly 130 compounds per adduct type. Within each adduct group, the specific compounds were chosen to span a representative range of molecular complexities, from small aromatic compounds to large bicyclic peptide-like structures, prioritizing visible structural diversity. This balanced design ensures that no single adduct dominates the aggregate results and that any differences in performance between prompt variants reflect genuine sensitivity to prompt content rather than imbalance in the test set.

3.5 Building the Core Inference Pipeline

With the dataset prepared and model weights available locally, the next step was implementing the core inference pipeline: the sequence of operations that transforms a user query into a CCS prediction.

This pipeline was built incrementally, starting from an exploratory script (*efficiency_tests.py*) used to establish a baseline configuration for model loading and basic response handling. The pipeline was then consolidated and extended in *prompt_tests.py*, which is the script used to generate the results reported in this work. Its final integration with the web interface is addressed later in this chapter.

3.5.1 Model and Tokenizer Loading

All loading logic is encapsulated in a single function, *load_model*, which handles tokenizer initialization, weight loading, precision configuration, and final preparation for inference. The function is called once at application startup and returns the ready-to-use (model, tokenizer) pair used by the rest of the pipeline.

The tokenizer is initialized using *AutoTokenizer.from_pretrained* with the local model path and *trust_remote_code=True*, which is required to support the custom tokenizer configuration used by the Qwen-based DeepSeek models. A necessary adjustment is explicitly setting the pad token to the end of sequence (EOS) token when no separate padding token is defined. Without this step, tokenization raises an error during input preparation.

Model weights were loaded using *AutoModelForCausalLM.from_pretrained*. Two parameters govern memory efficiency: *dtype* is set to *torch.float16* to reduce the memory footprint relative to the default float32 representation while preserving sufficient numerical fidelity for inference; and *low_cpu_mem_usage* is enabled to reduce peak RAM consumption during the loading phase. On the primary development machine, the *device_map* parameter is set to CPU, routing all computation to the processor. This precision configuration was the result of an empirical comparison against alternative quantization strategies, which is discussed in the next subsection.

Before returning the (model, tokenizer) pair, two final adjustments are applied. The call to *model.eval* switches the model to inference mode, disabling dropout and gradient tracking. Additionally, when the installed *PyTorch* version supports it, the model is wrapped with *torch.compile(model, mode="reduce-overhead")*, which produces an optimized computation graph that reduces per-call Python overhead and can noticeably speed up inference on CPU without altering the model's outputs.

3.5.2 Quantization Strategies Evaluated

Before settling on the configuration described above, two additional quantization approaches were explored. Dynamic int8 quantization via *PyTorch* was applied post-load (*torch.quantization.quantize_dynamic*), converting the weights of all nonlinear layers to

int8 at runtime, but it required the model to be loaded in float32 first since it is not compatible with float16 inputs on CPU. Weight-only int8 quantization via *torchao* (*Int8WeightOnlyConfig*, applied with *quantize*) stored the weights in int8 and dequantized them to float32 on the fly during matrix multiplication, and was directly compatible with float16 loading.

Both strategies successfully reduced the model's memory footprint and allowed stable execution within the 16 GB RAM constraint of the primary development machine. However, after comparing them empirically against a simpler configuration in which the model is loaded directly in *torch.float16* with no additional quantization step, this configuration was found to offer the best balance between memory usage, inference speed, and output quality for the models considered in this work. The final pipeline therefore retains *dtype=torch.float16* as the sole precision-reduction mechanism, and both quantization paths were removed from the production code and are not present in the final scripts of the repository.

3.5.3 Generation Parameters

All generation logic is encapsulated in a single function, *predict_ccs*, which receives the model and tokenizer together with the prompt and the supporting data. It tokenizes the prompt, runs the inference step, and returns the parsed prediction. The following paragraphs describe the generation step itself.

Inference is executed inside a *torch.inference_mode* context to disable gradient tracking and reduce memory overhead. Most decoding parameters were adapted to the size of the backbone model; one was held constant across variants.

The *do_sample* parameter depends on the model variant. The 1.5B variant uses greedy deterministic decoding (*do_sample=False*), which provides fully reproducible outputs. Meanwhile, the 7B variant uses sampling-based decoding (*do_sample=True*, *temperature=0.3*, *top_p=0.9*), since greedy decoding in DeepSeek-R1 distilled models tends to produce repetitive loops during long reasoning traces, a behavior explicitly discouraged by the DeepSeek-R1 official usage recommendations (Section 2.5.4). While these recommendations target a temperature range of 0.5-0.7 for open-ended reasoning

tasks, a lower value of 0.3 was adopted here because CCS prediction is a regression-like task with a single correct numerical answer per query (Section 2.6.1). The lower temperature preserves numerical precision while remaining stochastic enough to break the greedy-decoding loops. The *top_p* value was set to 0.9, slightly below the 0.95 suggested by DeepSeek, consistent with the restrictive temperature setting.

A *repetition_penalty* of 1.15 is applied uniformly across all three variants. This is a mild penalty within the typical 1.1-1.2 range discussed in Section 2.6.3 that reinforces the anti-loop behavior for the sampling variants and adds a light regularizing bias for the greedy variant, while keeping the decoding configuration identical across model sizes so that any observed differences in prediction quality can be attributed to the model itself rather than to a change in generation parameters.

The *max_new_tokens* parameter was set to a deliberately high value of 10000 tokens across all three model variants. The intent is purely permissive: the limit is set high enough that it is rarely reached in practice, so that the evaluation reflects the model's natural stopping behavior rather than an artificial truncation imposed by the generation budget. Using the same value across variants also keeps the experimental conditions identical when comparing prompts and model sizes.

Independently of the decoding strategy, *pad_token_id* is initialized to *tokenizer.eos_token_id* to prevent generation errors when the input is padded to the model's expected length.

3.5.4 Response Extraction and Numeric Parsing

Once the model has finished generating, the raw output must be converted into a single numeric CCS value. Its role is to interpret the free-form text emitted by the model so that the predicted value can be compared against the experimental reference.

Extraction proceeds in two steps inside *predict_ccs*. First, the newly generated text is isolated by decoding the full output and subtracting the decoded prompt, which avoids token-boundary artefacts. Then a dedicated parser, *parse_response*, locates the predicted number.

The parser was refined iteratively after inspecting model responses produced across the prompt-engineering experiments, and converged on a short cascade of three strategies, summarized in Table 2. Before matching, the text is normalized to remove markdown and LaTeX artefacts, and only the portion following the `</think>` tag is considered, so that intermediate values from the chain-of-thought reasoning trace do not contaminate the final prediction. Every candidate value is required to fall within the physically plausible CCS range (50-500 Å²); this filter caught real malformed outputs during testing, such as values emitted in scientific notation.

Table 2. Parsing cascade of *parse_response*

Num	Strategy	Captures
1	Final CCS: tag	The explicit format requested by the prompt.
2	<code>\boxed{}</code> wrapper	A <code>\boxed{<num>}</code> answer, occasionally emitted by DeepSeek-R1 as suggested by its own usage recommendations.
3	Last plausible number	A defensive last resort that picks the final in-range number from the post-think text.

If no strategy succeeds, the response is flagged as a parsing failure and excluded from the evaluation metrics, so that degenerate outputs do not bias the comparison between prompts. When extraction succeeds, the value is forwarded to a separate classifier, *classify_prediction*, described in the next section.

3.5.5 Prediction Classification

Once a numeric CCS value has been successfully extracted by *parse_response*, it is forwarded to a dedicated classifier, *classify_prediction*, whose role is to determine whether the value constitutes a genuine prediction or one of two failure patterns the classifier is designed to detect. The classifier does not modify the predicted value; it only attaches a categorical label so that the evaluation pipeline can later separate meaningful results from degenerate ones.

The classification logic is a short sequence of tolerance-based checks against two reference values: the mean CCS of the training set (*stats['ccs_avg']*) and the CCS values of the few-shot reference compounds provided in the prompt. Three exclusive categories are produced, summarized in Table 3.

Table 3. Categories produced by *classify_prediction*

Category	Condition	Interpretation
dataset_avg	Prediction within $\pm 0.5 \text{ \AA}^2$ of the training set mean	Mean anchoring: the model defaulted to a safe central value rather than committing to a specific estimate.
exact_copy	Prediction within $\pm 0.02 \text{ \AA}^2$ of any reference example CCS	Reference value copying: the model reproduced one of the few-shot examples instead of interpolating.
interpolated	Neither of the above	Genuine prediction: the value is distinct from both the dataset average and every reference example.

The order of the checks is significant. A prediction that happens to coincide both with the dataset mean and with a reference value is classified as *dataset_avg*, since mean anchoring is considered the more diagnostic failure mode. The tolerances were chosen empirically: $\pm 0.5 \text{ \AA}^2$ for the mean is wide enough to absorb the rounding behavior observed in small models, while $\pm 0.02 \text{ \AA}^2$ for reference copying is tight enough to identify only literal reproductions of an example value rather than coincidental matches.

The classifier serves two complementary purposes. During the prompt evaluation, it labels each prediction so that the quantitative behavior of each prompt variant can be analyzed in detail in Chapter 4. In the production application (*app.py*), the same labels are surfaced to the user as a confidence indicator, signaling whether the prediction is a genuine interpolation or one of the two diagnostic patterns.

3.6 Prompt Engineering

Prompt engineering is the central methodological contribution of this work. Building on the prompting principles of Section 2.5, the prompt was developed through four successive iterations, evaluated on a fixed 390-compound test bench (Section 3.4.5) with the script *prompt_tests.py*.

All four iterations share a common skeleton, so that differences in predictive behavior reflect only the content added on top of it. The skeleton has four elements:

- Role assignment and task statement: it establishes the model as an ion mobility mass spectrometry expert and asks it to predict the CCS of the input molecule from its SMILES and adduct.
- Numbered procedural recipe: steps to reach an estimate: read the SMILES, classify the structure as compact or extended, apply the adduct correction, and output a value in the typical 100-300 Å² range.
- Molecule block: target SMILES and normalized adduct, with a short qualitative description from the *ADDUCT_INFO* dictionary (Section 3.4.2).
- Output directive and forced reasoning opening: requires the response to end with Final CCS: <number>, immediately followed by an opening <think> tag and the first words of a reasoning stem, which forces the model to enter the reasoning phase before producing the final numerical answer.

Each iteration extends this skeleton with specialized content blocks inserted between the role assignment and the molecule block. The next subsections describe what each variant adds and the rationale behind it.

3.6.1 Iteration 1 — Baseline Prompt: Molecular Descriptors Only

The first prompt variant (*build_prompt_simple*) contains only the shared skeleton described above, with no additional content. No expert knowledge, no reference examples, and no domain-specific rules are included. This variant serves as the lower-bound baseline, establishing the model's numerical estimation capability under minimal context.

3.6.2 Iteration 2 — Adding Expert Chemical Rules

The second variant (*build_prompt_expert*) extends the baseline prompt by inserting an expert knowledge block between the role assignment and the instructions. The block consists of one introductory sentence followed by four structural factors that influence CCS in the gas phase:

- Molecular size and rigidity.
 - Compact, rigid structures (fused aromatic systems, polycyclic cores) yield lower CCS for their mass.
 - Extended, flexible structures (long aliphatic chains, single-bond rotations) yield higher CCS for their mass.
- Branching and substitution.
 - Branched molecules pack more compactly than linear isomers of the same mass, yielding lower CCS.
 - Bulky substituents (long side chains, multiple rings off a central atom) increase CCS.
- Heteroatoms and functional groups.
 - Heteroatoms (N, O, S) and halogens add mass but contribute relatively little to molecular volume.
 - Polar groups (–OH, –NH₂, –COOH) can promote intramolecular interactions, slightly reducing CCS.
- Adduct effect.
 - The adduct alters charge state and ion geometry.
 - The qualitative effect described in the molecule line below is applied as a small final adjustment.

These four directives translate the structural determinants of CCS established in Section 2.1.2 into natural-language rules that the model can apply during reasoning, compensating for the lack of fine-tuning by adding knowledge directly into the context.

3.6.3 Iteration 3 — Adding Few-Shot Reference Examples

The third variant (*build_prompt_examples*) extends the baseline by inserting a reference molecules block listing five compounds retrieved from the training dataset, selected through the structural similarity procedure described in Section 3.4.3. Each example is formatted on three lines (SMILES, adduct, experimental CCS), and the five examples are presented as a block immediately after the role assignment.

The procedural recipe is also extended with one additional step, instructing the model to compare the target compound with the reference examples and use their

experimental CCS values as anchors when producing the final estimate. The number of examples per prompt was chosen as a balance between providing enough numerical grounding for interpolation and keeping the prompt length manageable for the small backbone models targeted by this work.

3.6.4 Iteration 4 — Full Combined Prompt with Heuristic Anchor

The fourth and final variant (*build_prompt_combined*) integrates all the previous components (expert knowledge and few-shot reference examples). The procedural steps used are the same five-step version in the third iteration, which includes the comparison-with-references step.

No further content is added beyond what is already present in the two source iterations. This variant tests the hypothesis that domain knowledge and few-shot anchoring are complementary: the expert rules give the model a principled framework for reasoning about the structure, while the reference examples constrain the numerical output to a region of the CCS space that is consistent with experimentally observed compounds of similar structure.

3.6.5 Final Prompt Structure

The four variants were compared under identical conditions (same model, generation parameters, and retrieval procedure). Iteration 4 (*build_prompt_combined*) was selected as the final configuration based on its overall accuracy and the lower incidence of failure modes. The quantitative comparison is reported in Chapter 4.

3.7 Hardware Configuration and Execution Setup

The system was developed and evaluated across two CPU-only environments: a personal laptop with 16 GB of system RAM, used for day-to-day development and for running the smaller backbone variant, and a university workstation with 32 GB of system RAM, used to evaluate the larger variants and to generate the final results reported in Chapter 4.

GPU acceleration was not viable in either environment. The discrete GPUs available (2 GB of VRAM on the laptop, 6 GB on the university workstation) are not large enough to hold the weight tensors of the DeepSeek-R1-Distill-Qwen models at the precision required for stable inference, even for the 1.5B variant in float16. Rather than maintain a parallel GPU code path that would never be exercised in practice, the project commits to a uniform CPU-only execution model. This decision also has a methodological benefit: the same execution strategy is used for the 1.5B and 7B variants, so that differences in predictive behavior between model sizes can be attributed to the model itself rather than to a change in the runtime stack.

A consistent set of memory optimization is applied across all three variants: weights are loaded in float16 with `low_cpu_mem_usage=True`, and inference is executed inside a `torch.inference_mode` context as already described in Section 3.5.3. Earlier exploratory work with dynamic int8 quantization (both *PyTorch* built-in `quantize_dynamic` and the *torchao* backend) was discarded after preliminary tests showed unstable behavior and no clear latency benefit on CPU for these model families. Float16 with the lazy-loading flag therefore serves as the single, uniform memory configuration.

The personal laptop's 16 GB budget is sufficient only for the 1.5B variant, which is the configuration used during local development and during testing of the Flask application described in Section 3.8. The university workstation's 32 GB budget enables evaluation of the 7B variant, which required adjustments to the generation parameters as described in Section 3.5.3. The bulk of the prediction-quality results reported in Chapter 4 were generated on the university workstation.

3.8 Web Application Development

Once the core inference pipeline had been consolidated through the prompt evaluation experiments (Section 3.6), it was integrated into a Flask web application (*app.py*) that exposes the functionality through a REST API consumed by a browser-based user interface. The application is designed to run locally on a single machine, with no external service dependencies beyond the locally stored model weights and the preprocessed *train.csv*.

3.8.1 Backend Architecture and Initialization

The application follows a single-file Flask structure. At startup, the *initialize_app* function executes a deterministic initialization sequence. It first loads and parses the *train.csv* (46,391 entries), then computes the dataset-wide CCS statistics needed by the classifier (minimum, maximum, and mean) and finally loads the tokenizer and model weights from a local path. Only when all three steps have been completed successfully does the Flask server begin accepting requests, ensuring that the application never serves predictions with an uninitialized model.

Model loading reuses the configuration of the evaluation scripts (Section 3.5.3). The weights are loaded onto CPU in float16 precision with *low_cpu_mem_usage=True*, and GPU visibility is explicitly disabled via the *CUDA_VISIBLE_DEVICES* environment variable to prevent accidental device fallback on machines where a small GPU is present but unsuitable for inference. The compiled model graph is then wrapped with *torch.compile(mode="reduce-overhead")* to reduce per-call dispatch latency at runtime without altering the predicted values.

The training dataset is loaded into memory in full as a list of Python dictionaries. For the 46,391-entry training set used in this work, this representation enables a single linear scan to support both exact-match lookup and similarity-based example retrieval (Section 3.4.3), with negligible cost relative to model inference time.

3.8.2 REST API: /predict and /status Endpoints

The /predict endpoint accepts POST requests with a JSON body containing the smiles and adduct fields. Before invoking the language model, the request handling logic performs a short-circuit check against the in-memory training dataset. If an exact match for the submitted SMILES and normalized adduct is found, the experimentally measured CCS value is returned directly with a *from_dataset* flag. This ensures that compounds present in the training set always return their experimental value with zero inference cost.

For unseen compounds, the full pipeline is executed. It performs similarity-based selection of five reference examples (Section 3.4.3), constructs the combined prompt from Section 3.6.4, runs model inference and response parsing (Sections 3.5.3-3.5.4), and

finally applies prediction classification (Section 3.5.5). The JSON response includes the predicted CCS value, the prediction type label produced by the classifier, a textual reasoning summary, the extracted molecular features of the input SMILES, and the top three reference compounds used during retrieval, which is enough information for the frontend to render a complete result panel without further requests.

A lightweight `/status` endpoint returns a JSON summary of the application's runtime state, including whether the model and dataset are loaded, the size of the training set in memory, and the computed dataset statistics. This endpoint was used during development to verify that the initialization sequence had completed correctly before submitting prediction requests and could serve as a endpoint in a production deployment.

3.8.3 Frontend interface

The browser-based frontend (`templates/Index.html`) is a single-page application served by Flask's template engine. The user submits a query through a three-field form (SMILES, adduct type, and a predict button), with one example SMILES per supported adduct drawn from the test set, accessible as one-click shortcuts for quick testing. Predictions are fetched through the browser Fetch API without page reloads.

The results panel surfaces the model's output in four blocks. They display the predicted CCS value, the prediction metadata, a textual reasoning summary, and the top reference compounds used during inference. An additional block for error messages is shown when relevant. The internal prediction-type labels produced by the classifier (interpolated, exact_copy, dataset_avg) drive the metadata and reasoning fields. The metadata includes a method badge that classifies the result at a high level (exact match for dataset hits, AI inference for model predictions, heuristic for parser fallbacks), while the confidence and reasoning output reflect the specific classification outcome: High for exact dataset matches, Medium for genuine model interpolations, and Low for reference-value copies, mean-anchored outputs, and parser fallbacks. The visual styling uses a light two-column layout with a neutral and simple theme.

3.9 System Refinement and Final Integration

The final phase of implementation involved consolidating the findings from all previous stages into a coherent production system, resolving edge cases identified during end-to-end testing, and verifying pipeline behavior across the full evaluation test bench.

3.9.1 Consolidation of Prompt and Parsing Logic into the Application

The initial experimentation conducted in *efficiency_tests.py*, which evolved into *prompt_tests.py* producing a stable final prompt configuration and a robust parsing cascade. These were ported into *app.py* as the production implementations of *build_prompt* and *predict_ccs* respectively, with no modifications required beyond the import-path adjustments needed to integrate them into the Flask application.

3.9.2 Interface Improvements and User Experience Refinements

Several refinements were made to the frontend based on end-to-end testing. The results panel was extended to include a confidence badge distinguishing between exact database matches, AI-generated predictions, and heuristic fallback values, providing immediate feedback on the reliability of each prediction. The adduct selector was populated with the three most common adduct types present in the training dataset.

3.9.3 End-to-End Validation on the Compound Test Bench

A final end-to-end validation pass was conducted using the reference compounds, executing the complete pipeline from HTTP request submission through prompt construction, model inference, output parsing, and JSON response assembly. The validation confirmed that the prediction pipeline completed successfully for all test bench compounds without triggering fallback, except for a small number of cases in the 1.5B model where the generation budget (*max_new_tokens=10000*) was exhausted before the model emitted a parseable CCS value. Aside from those cases, all predicted CCS values fell within the physically plausible range, and the similar compounds panel correctly surfaced structurally relevant training entries for each query.

4 RESULTS

This chapter reports the experimental evaluation of the system described in Chapter 3. It is organized in three blocks, covering quantitative accuracy for each model and prompt variant, prediction quality and systematic failure modes, and the final integrated system in the web application. All quantitative results in Sections 4.2 and 4.3 were obtained by running *prompt_tests.py* on the 390-compound test bench (Section 3.4.5), while Section 4.4 demonstrates the integrated system through *app.py*.

4.1 Evaluation Setup

This section summarizes only the metrics used to report the results, as the test bench (Section 3.4.5), the models evaluated (Sections 3.3 and 3.7) and the four prompt variants (Section 3.6) have been introduced in their respective places.

Prediction accuracy is measured against the experimental CCS values of the 390-compound test bench using six complementary metrics.

- Mean Squared Error (MSE): serves as the basis for the RMSE and is not reported separately in the result tables.
- Root Mean Squared Error (RMSE): square root of the MSE, expressed in the same units as the CCS ($\text{\AA}^2$). Because squaring penalizes large errors more heavily, the RMSE is more sensitive to outlier predictions (reference-value copying).
- Mean Absolute Error (MAE): reports the average deviation between predicted and experimental CCS values in absolute units ($\text{\AA}^2$). It is more robust to outliers than the RMSE and serves as the primary metric of the evaluation.
- Mean Absolute Percentage Error (MAPE): expresses the same deviation in relative terms, which is more interpretable across the range of the test bench.
- Median Absolute Percentage Error (MedAPE): complements the MAPE by using the median instead of the mean, it is less sensitive to the large errors introduced by degenerate predictions and reflects better the accuracy of the system.
- Coefficient of determination (R^2): ranges from $-\infty$ to 1, where 1 indicates perfect prediction, 0 means the model performs no better than predicting the dataset mean, and negative values indicate worse performance than that baseline.

4.2 Quantitative Accuracy Results

This section reports the accuracy of each model and prompt combination over the 390-compound test bench, evaluated under the metrics defined in Section 4.1. The 1.5B and 7B models are presented separately to highlight the prompt sensitivity of each one, and a cross-model comparison closes the section. The four prompt variants are evaluated under identical generation parameters, so any difference between them reflects the effect of the prompt content alone. Tables 4 and 5 summarize the performance of the DeepSeek-R1-Distill-Qwen models across the prompt variants introduced in Section 3.6.

4.2.1 Results for the 1.5B model

Table 4. Accuracy of the 1.5B model across the four prompt variants, evaluated on the 390-compound test bench. The best value in each column is highlighted in bold.

Prompt variant	RMSE	MAE	MAPE	MedAPE	R ²
Iteration 1 – Simple	59.33 Å ²	47.30 Å ²	24.84 %	20.83 %	0.00116
Iteration 2 – Expert knowledge	61.17 Å ²	52.19 Å ²	27.91 %	26.01 %	0.00018
Iteration 3 – Few-shot examples	17.07 Å²	11.37 Å²	5.97 %	4.37 %	0.31731
Iteration 4 – Combined	20.74 Å ²	12.89 Å ²	6.81 %	4.1 %	0.21607

The 1.5B model results reveal a sharp contrast between configurations with and without few-shot examples. Iterations 1 and 2 yield MAE values around 50 Å² and R² values near zero, indicating virtually no predictive signal beyond the dataset mean, with the expert-knowledge block producing no measurable improvement over the simple baseline. The introduction of few-shot examples reduces the MAE by approximately a factor of four and brings the MAPE below 10%, with a MedAPE approximately 4.4% confirming that the typical prediction is considerably more accurate than the mean error suggests and that residual error is driven by a minority of outlier cases. Within the two top-performing prompts, iteration 3 slightly outperforms iteration 4, suggesting that the additional expert-knowledge context dilutes the influence of the numerical anchors rather than complementing them.

4.2.2 Results for the 7B model

Table 5. Accuracy of the 7B model across the four prompt variants, evaluated on the 390-compound test bench. The best value in each column is highlighted in bold.

Prompt variant	RMSE	MAE	MAPE	MedAPE	R ²
Iteration 1 – Simple	53.77 Å ²	43.25 Å ²	22.91 %	20.08 %	0.01070
Iteration 2 – Expert knowledge	43.97 Å ²	34.20 Å ²	17.92 %	13.40 %	0.04028
Iteration 3 – Few-shot examples	14.24 Å ²	9.47 Å ²	5.02 %	3.61 %	0.44142
Iteration 4 – Combined	12.88 Å²	8.67 Å²	4.56 %	3.36 %	0.50445

The 7B model follows a broadly similar pattern but with two notable differences. The expert-knowledge block alone (iteration 2) produces a measurable improvement over the simple baseline, indicating that the larger model is able to translate qualitative chemical rules into a corrective signal even without numerical anchors, a capability absent in the 1.5B. The combined prompt (iteration 4) is the best overall configuration, achieving the lowest MAE (8.67 Å²), the lowest MedAPE (3.36%), and an R² of 0.50445, confirming that expert knowledge and few-shot examples behave as complementary sources of information that the 7B model integrates coherently.

4.2.3 Cross-model comparison

Comparing the two models across the four prompt variants reveals two consistent patterns. The first is the outperformance of the 7B model over the 1.5B across all prompt configurations. The widest gap appears under the expert-knowledge prompt, where the MAE difference exceeds 15 Å², reflecting the larger model's superior ability to apply qualitative chemical reasoning. The narrowest gap appears under the few-shot prompts, where both models converge toward MAE values in the 9-11 Å² range and MedAPE values below 5%, suggesting that numerical anchors largely compensate for the capacity difference. Additionally, R² values near zero for the 1.5B model and low but meaningful for the 7B in configurations without reference examples confirm that predictions without few-shot anchors offer little improvement over the dataset mean.

The second pattern concerns which prompt works best for each model. The 1.5B performs slightly worse under the combined prompt than with few-shot alone, whereas the 7B achieves its best results with it, reaching the lowest MAE and the only R^2 above 0.5 across all configurations. This confirms that the ability to integrate expert knowledge and reference examples coherently is a capacity that emerges with model scale. Section 5.1 discusses this in more detail.

Figure 4 illustrates these patterns jointly, plotting predicted against experimental CCS for all eight model-prompt combinations. Darker-toned markers correspond to the more complex prompt variants (few-shot and combined), with their higher concentration near the diagonal reflecting the improved prediction accuracy of these configurations. The wider dispersion of the 1.5B configurations and the tighter clustering of the 7B few-shot and combined variants around the diagonal are consistent with the MAE differences reported in Tables 4 and 5.

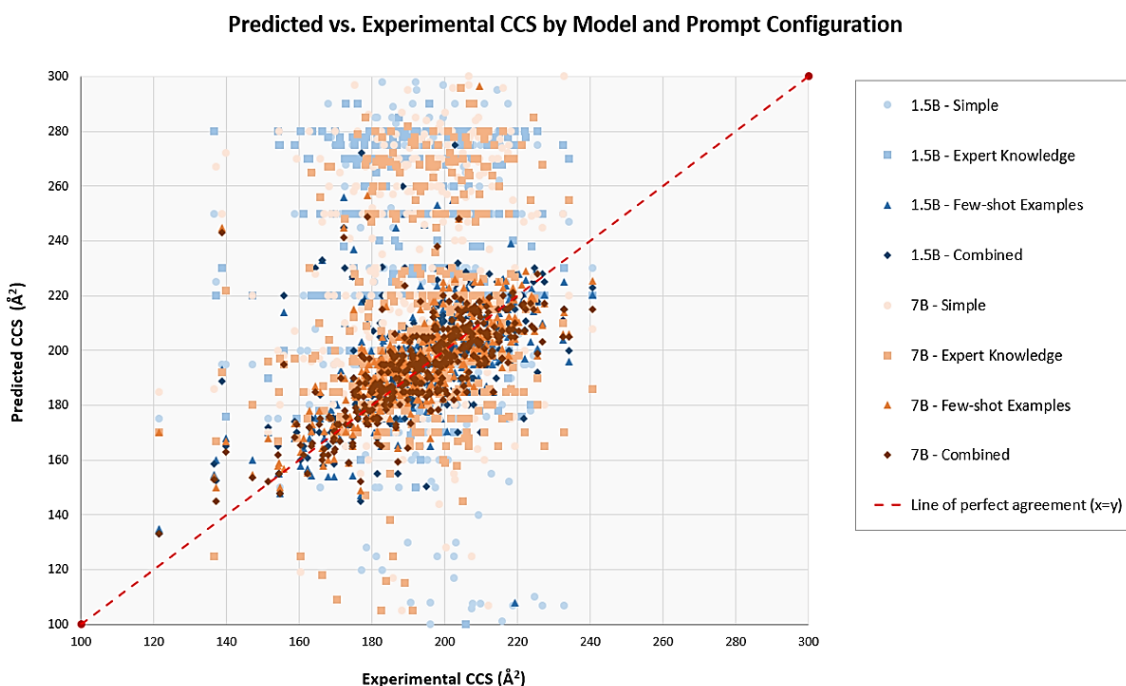


Figure 4. Predicted vs. experimental CCS values for the DeepSeek-R1-Distill-Qwen-1.5B and 7B models across the four prompt variants, evaluated on the 390-compound test bench. The dashed red line represents perfect prediction ($x = y$).

4.3 Prediction Quality Analysis

Beyond raw accuracy, every prediction produced during the evaluation was labelled with one of the four categories defined by the classifier introduced in Section 3.5.5 (interpolated, exact copy, dataset average, heuristic fallback). Tables 6 and 7 summarize the resulting distribution for the 1.5B and 7B models respectively.

Table 6. Distribution of prediction categories for the 1.5B model across the four prompt variants, evaluated on the 390-compound test bench.

Prompt variant	Interpolated	Exact copy	Dataset avg	Fallback
Iteration 1 – Simple	389 (99.7 %)	0 (0.0 %)	0 (0.0 %)	1 (0.3 %)
Iteration 2 – Expert knowledge	390 (100.0 %)	0 (0.0 %)	0 (0.0 %)	0 (0.0 %)
Iteration 3 – Few-shot examples	320 (82.1 %)	66 (16.9 %)	4 (1.0 %)	0 (0.0 %)
Iteration 4 – Combined	318 (81.5 %)	66 (16.9 %)	5 (1.3 %)	1 (0.3 %)
Total from all iterations	1417 (90.8 %)	132 (8.5 %)	9 (0.6 %)	2 (0.1 %)

Table 7. Distribution of prediction categories for the 7B model across the four prompt variants, evaluated on the 390-compound test bench.

Prompt variant	Interpolated	Exact copy	Dataset avg	Fallback
Iteration 1 – Simple	385 (98.7 %)	0 (0.0 %)	5 (1.3 %)	0 (0.0 %)
Iteration 2 – Expert knowledge	383 (98.2 %)	0 (0.0 %)	7 (1.8 %)	0 (0.0 %)
Iteration 3 – Few-shot examples	325 (83.3 %)	57 (14.6 %)	8 (2.1 %)	0 (0.0 %)
Iteration 4 – Combined	350 (89.7 %)	31 (8.0 %)	9 (2.3 %)	0 (0.0 %)
Total from all iterations	1443 (92.5 %)	88 (5.6 %)	29 (1.9 %)	0 (0.0 %)

4.3.1 Distribution of prediction categories across prompt variants

Tables 6 and 7 trace a coherent picture of how each prompt variant shapes model behavior. In the prompts without reference examples, virtually every prediction is classified as a model interpolation, since exact-copy detection requires reference examples to be present in the prompt. The introduction of few-shot examples changes the picture substantially, with exact-copy predictions emerging as a sizable fraction of the output and the proportion of pure interpolations dropping accordingly.

4.3.2 Reference value copying as the primary failure mode

When few-shot examples are present, reference-value copying becomes the most frequent failure mode. In the 1.5B model it reaches 16.9% in both few-shot and combined prompts, while in the 7B model the rates are 14.6% and 8.0%. In the 7B model, the combined prompt reduces copying by half compared to few-shot. This mirrors the accuracy results of Section 4.2.

4.3.3 Mean anchoring across model sizes

Mean anchoring is marginal overall but shows a consistent difference between models. While essentially absent in the 1.5B model predictions, it accounts for almost 2% of predictions in the 7B model across all four prompt variants.

4.3.4 Parsing fallback rate

Parsing fallbacks are negligible. Across the 3,120 predictions of this evaluation, only 2 trigger the fallback path, both in the 1.5B model. These two cases correspond to outputs in which the 1.5B model failed to emit a parseable CCS value before exhausting the 10,000-token generation budget, which was set deliberately high (Section 3.5.3) precisely to prevent this kind of premature truncation. The structured output directive is therefore reliable across model sizes and prompt configurations, and the fallback path remains in the pipeline only as a safeguard.

4.4 Web Application Demonstration

This section complements the quantitative results of Sections 4.2 and 4.3 by illustrating the integrated system in operation. Two representative snapshots of the web interface served by *app.py* and displayed through *Index.html* are presented, covering the initial loading state (Figure 5) and a completed prediction (Figure 6), to illustrate how the pipeline described in Section 3.8 delivers its outputs to the end user.

4.4.1 Initial state and query submission

On initial load, the left column displays the input form with an empty SMILES field, the adduct selector defaulted to $[M+H]^+$, and the optional example chips

immediately below the input. The right column shows an empty-state placeholder reminding the user that no prediction has been requested yet. The Header reports the active model variant, the size of the in-memory training corpus, and the runtime configuration, which lets the user confirm that the backend has completed its initialization sequence (Section 3.8.1) before submitting any query.

Figure 5. Initial state of the web interface.

Clicking one of the example chips populates both the SMILES field and the adduct selector with a single test-set compound for the corresponding adduct, which is the fastest way to obtain a representative result without manually typing a structure. Once the form is populated, clicking Predict submits the query to the /predict endpoint and triggers the full inference pipeline described in Section 3.8.2, after which the results panel populates as described in the following subsection.

4.4.2 Completed prediction

Once the backend returns a prediction, the results column populates with four ordered blocks. The predicted CCS value is presented first in a large display, followed by the prediction metadata block then reports the method label produced by the classifier (Section 3.5.5), the associated confidence indicator, and the adduct used for the query.

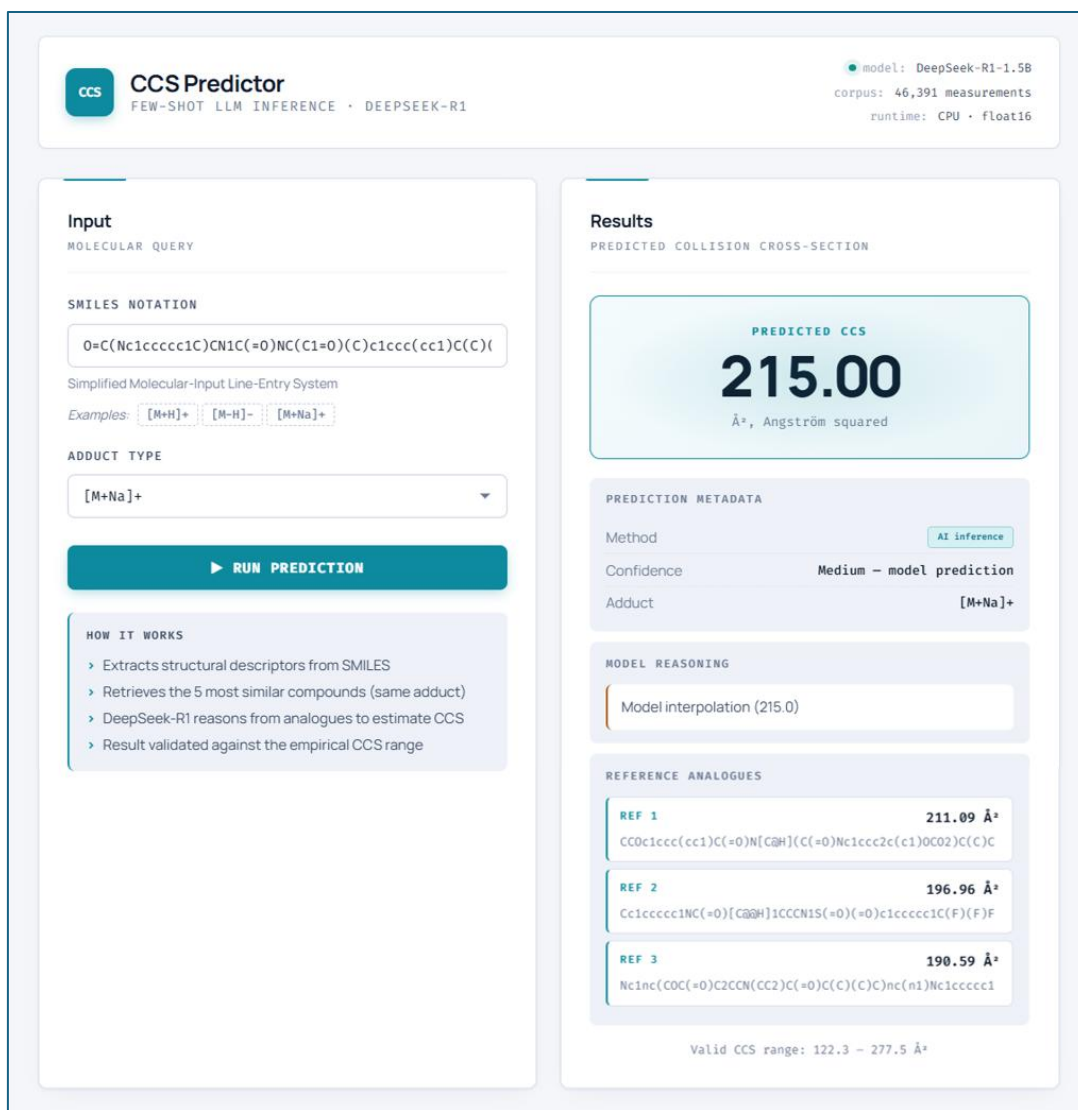


Figure 6. Web interface after a successful prediction, showing the four output blocks.

Below the metadata, the model reasoning block exposes the conclusion extracted from the textual justification emitted by the language model. The reference analogues block closes the panel, listing the top compounds returned by similarity-based retrieval (Section 3.4.3) together with their experimentally measured CCS values, so that the user can directly compare the predicted value against the ones used to anchor it.

5 DISCUSSION

5.1 Interpretation of Qualitative Results

The baseline prompt variants, which contain no reference examples, produce R^2 values close to zero in both model sizes. This means the predictions are no better than simply guessing the dataset mean for every compound, which would be the worst result imaginable. Without seeing any experimental CCS values in the prompt, the model has no value to work from. It can describe the molecular structure and reason about its shape, but it has no way of translating that reasoning into a specific number on the CCS scale.

When few-shot examples are added in iteration 3, this changes immediately. The model now has real measured CCS values for structurally similar compounds right in front of it, and it can use them as reference points to estimate where the target should fall. This is what drives the sharp drop in MAE, not the extra context or the longer prompt, but the presence of actual numbers the model can reason around.

The most interesting finding concerns the difference in the response of the two models to the addition of the expert knowledge block. Adding qualitative chemical rules to the prompt improves the 7B model noticeably, while the 1.5B model shows no improvement at all with the same prompt. The most likely explanation is that reading a qualitative chemical rule and using it to produce a numerical correction are very different things. The 7B model can do this to some degree, the 1.5B simply cannot.

This difference also explains what happens with the combined prompt. The 7B model benefits from having both rules and examples at the same time, and its predictions improve further. The 1.5B model, on the other hand, seems to get confused by the extra content. With more text to process and less capacity to integrate it, the combined prompt fails to reduce reference-value copying, suggesting that the additional expert-knowledge content dilutes the influence of the numerical anchors rather than complementing them, which is the opposite of what the combined prompt is trying to achieve.

Taken together, these patterns suggest that the system works best when the model has both a numerical frame of reference and enough capacity to reason around it, and that these two conditions are not always met simultaneously at the scales evaluated.

5.2 Prompt Engineering vs. Machine Learning

The overall R^2 ceiling, which reaches only 0.204 even in the best configuration, points to a deeper issue. The model was never trained on CCS data, so it has no internal sense of how CCS values are distributed across different types of molecules, and a handful of retrieved examples cannot fully substitute for that kind of learned knowledge. Systems like DeepCCS or graph neural network models are trained directly on experimental CCS data, and they learn the relationship between molecular structure and CCS from thousands of examples. These systems probably achieve considerably lower prediction errors than those observed in this work, and for real analytical use that gap is too large to ignore.

The reason this work takes a prompt engineering approach rather than a machine learning one is not to compete on accuracy, but to explore what general-purpose reasoning models can do without any previous training. The knowledge within the prompt can be updated or corrected at any time and the reasoning trace for each prediction helps determine the source of the given value. Classical machine learning models for CCS do not offer either of these things, which may be worthwhile depending on the use case.

5.3 System Limitations

Several limitations were identified during development and evaluation of this work, from which three are highlighted. The first limitation observed was the molecular feature extractor, which relies on character-level SMILES parsing rather than a proper chemical library, a deliberate simplicity trade-off noted in Section 3.4.1. This can produce incorrect descriptor counts for non-standard molecular representations. Another limitation was the adduct coverage, which was restricted to the three types present in the METLIN-CCS dataset, preventing the system from handling compounds measured under other ionization modes.

The most consequential limitation was the model capacity. Both model sizes showed systematic failure modes under few-shot prompting that reflect the limits of what these architectures can achieve through prompting alone. The consistent improvement from 1.5B to 7B suggests that larger models would raise those limits, but closing the gap with dedicated machine learning systems would likely require either significantly larger models or some degree of fine-tuning.

5.4 Future Work

Several directions derive from the limitations described above. Replacing the SMILES parser with RDKit would improve descriptor accuracy and retrieval quality with minimal changes to the rest of the pipeline.

Expanding the CCS dataset would serve two purposes at once. The larger reference pool would improve coverage of underrepresented compound classes while including examples for additional ionization modes, allowing the system to support a wider range of adduct types. A larger test bench would also strengthen the reliability of the results, since the current 390-compound benchmark is small enough that individual compounds can have a disproportionate effect on overall metrics.

On the computational side, access to a cloud-based GPU would solve the latency problem and open the door to evaluating larger models. For more demanding workloads, paid cloud services would provide the additional resources and stability needed to evaluate even larger model variants.

The 14B checkpoint was downloaded and prepared during the project but ultimately not evaluated. Running a model of that size on CPU would have taken too long to complete with the full test bench and, given that the results from the 1.5B and 7B variants were already sufficient to draw meaningful conclusions about how prediction quality scales with model size, it was not considered essential for this work. It remains a natural next step for future evaluation, ideally on a GPU-enabled environment.

Finally, a direct comparison against a retrained machine learning baseline on the same test bench would give a clearer picture of how much accuracy is lost by avoiding task-specific training altogether. Beyond that, exploring whether lightweight fine-tuning of the language model on CCS data could narrow that gap is perhaps the most interesting open question this work leaves behind, since it would combine the flexibility and interpretability of the LLM approach with the distributional knowledge that only training on real data can provide.

6 CONCLUSIONS

This work set out to explore whether a reasoning-oriented language model, deployed entirely on consumer hardware and guided by a carefully designed prompting strategy, could produce meaningful CCS predictions for small organic molecules. The answer is a qualified yes, with room for future improvements.

All specific objectives were met. The theoretical background covering CCS, ion mobility mass spectrometry, SMILES notation, and the relevant aspects of large language models were reviewed and integrated into the system design. A full inference pipeline was implemented, from SMILES input to numerical prediction, including similarity-based retrieval, prompt construction, model inference, and response parsing. Four prompt variants of increasing complexity were systematically evaluated on a balanced 390-compound benchmark. The impact of model scale was assessed by comparing the 1.5B and 7B variants under identical conditions, and the resulting system was deployed as a functional web application that returns a predicted CCS value.

The evaluation showed that prompt design has a substantial effect on prediction quality, and that the most critical component is the inclusion of few-shot reference examples retrieved from experimental data. Without them, neither model produces predictions meaningfully better than the dataset mean, while their inclusion reduces the error by a factor of three. The results also showed that the ability to benefit from expert chemical knowledge grows with model size, pointing to a clear direction for future work as larger models become more accessible.

This work contributes something that existing CCS prediction systems do not offer, which is a prediction accompanied by a confidence indicator that tells the user whether the result is a genuine interpolation or a known failure mode. The system also runs on hardware available to any individual researcher, without cloud dependencies or specialized infrastructure, which lowers the barrier to adoption significantly.

The approach is not a replacement for dedicated machine learning models but a complementary one, trading some predictive performance for transparency and accessibility. As language models continue to improve, the gap in accuracy is likely to narrow, making this kind of approach increasingly practical for real analytical use.

7 REFERENCES

- [1] Moses, T., & Burgess, K. (2023). Right in two: capabilities of ion mobility spectrometry for untargeted metabolomics. *Frontiers in Molecular Biosciences*, 10, 1230282. <https://doi.org/10.3389/fmolb.2023.1230282>
- [2] Zhang, X., Li, Q., Xu, Z., & Dou, J. (2020). Mass spectrometry-based metabolomics in health and medical science: a systematic review. *RSC Advances*, 10(6), 3092–3104. <https://doi.org/10.1039/c9ra08985c>
- [3] Zheng, X., Aly, N. A., Zhou, Y., Dupuis, K. T., Bilbao, A., Paurus, V. L., Orton, D. J., Wilson, R., Payne, S. H., Smith, R. D., & Baker, E. S. (2017). A structural examination and collision cross section database for over 500 metabolites and xenobiotics using drift tube ion mobility spectrometry. *Chemical Science*, 8(11), 7724–7736. <https://doi.org/10.1039/c7sc03464d>
- [4] Dodds, J. N., & Baker, E. S. (2019). Ion Mobility Spectrometry: fundamental concepts, instrumentation, applications, and the road ahead. *Journal of the American Society for Mass Spectrometry*, 30(11), 2185–2195. <https://doi.org/10.1007/s13361-019-02288-2>
- [5] May, J. C., & McLean, J. A. (2014). Ion Mobility-Mass Spectrometry: Time-Dispersive Instrumentation. *Analytical Chemistry*, 87(3), 1422–1436. <https://doi.org/10.1021/ac504720m>
- [6] Zhou, Z., Luo, M., Chen, X., Yin, Y., Xiong, X., Wang, R., & Zhu, Z. (2020). Ion mobility collision cross-section atlas for known and unknown metabolite annotation in untargeted metabolomics. *Nature Communications*, 11(1), 4334. <https://doi.org/10.1038/s41467-020-18171-8>
- [7] Paglia, G., Williams, J. P., Menikarachchi, L., Thompson, J. W., Tyldesley-Worster, R., Halldórsson, S., Rolfsson, O., Moseley, A., Grant, D., Langridge, J., Palsson, B. O., & Astarita, G. (2014). Ion mobility derived collision cross sections to support metabolomics applications. *Analytical Chemistry*, 86(8), 3985–3993. <https://doi.org/10.1021/ac500405x>
- [8] Colby, S. M., Thomas, D. G., Nuñez, J. R., Baxter, D. J., Glaesemann, K. R., Brown, J. M., Pirrung, M. A., Govind, N., Teeguarden, J. G., Metz, T. O., & Renslow, R. S. (2019b). ISICLE: A Quantum Chemistry Pipeline for establishing in silico collision cross section libraries. *Analytical Chemistry*, 91(7), 4346–4356. <https://doi.org/10.1021/acs.analchem.8b04567>
- [9] Guo, R., Zhang, Y., Liao, Y., Yang, Q., Xie, T., Fan, X., Lin, Z., Chen, Y., Lu, H., & Zhang, Z. (2023). Highly accurate and large-scale collision cross sections prediction with graph neural networks. *Communications Chemistry*, 6(1), 139. <https://doi.org/10.1038/s42004-023-00939-w>
- [10] Li, X., Wang, H., Jiang, M., Ding, M., Xu, X., Xu, B., Zou, Y., Yu, Y., & Yang, W. (2023). Collision cross section prediction based on machine learning. *Molecules*, 28(10), 4050. <https://doi.org/10.3390/molecules28104050>
- [11] Brown, T. B., Mann, B., Ryder, N., Subbiah, M., Kaplan, J., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., Agarwal, S., Herbert-Voss, A., Krueger, G., Henighan, T., Child, R., Ramesh, A., Ziegler, D. M., Wu, J., Winter, C., . . . Amodei, D. (2020). Language Models are Few-Shot Learners. *arXiv (Cornell University)*, 33, 1877–1901. <https://doi.org/10.48550/arxiv.2005.14165>
- [12] Guo, D., Yang, D., Zhang, H., Song, J., Wang, P., Zhu, Q., Xu, R., Zhang, R., Ma, S., Bi, X., Zhang, X., Yu, X., Wu, Y., Wu, Z. F., Gou, Z., Shao, Z., Li, Z., Gao, Z., Liu, A., . . . Zhang, Z. (2025). DeepSeek-R1 incentivizes reasoning in LLMs through reinforcement learning. *Nature*, 645(8081), 633–638. <https://doi.org/10.1038/s41586-025-09422-z>
- [13] Rudin, C. (2019). Stop explaining black box machine learning models for high stakes decisions and use interpretable models instead. *Nature Machine Intelligence*, 1(5), 206–215. <https://arxiv.org/abs/1811.10154>
- [14] Plante, P., Francovic-Fontaine, É., May, J. C., McLean, J. A., Baker, E. S., Laviolette, F., Marchand, M., & Corbeil, J. (2019). Predicting ion mobility collision Cross-Sections using a deep neural network: DeepCCS. *Analytical Chemistry*, 91(8), 5191–5199. <https://doi.org/10.1021/acs.analchem.8b05821>
- [15] May, J. C., Goodwin, C. R., Lareau, N. M., Leaptrot, K. L., Morris, C. B., Kurulugama, R. T., Mordehai, A., Klein, C., Barry, W., Darland, E., Overney, G., Imatani, K., Stafford, G. C., Fjeldsted, J. C., & McLean, J. A. (2014). Conformational ordering of biomolecules in the gas phase: nitrogen collision cross sections measured on a prototype high resolution Drift Tube Ion Mobility-MaSS spectrometer. *Analytical Chemistry*, 86(4), 2107–2116. <https://doi.org/10.1021/ac4038448>
- [16] Morris, C. B., Poland, J. C., May, J. C., & McLean, J. A. (2019). Fundamentals of Ion Mobility-Mass Spectrometry for the analysis of biomolecules. *Methods in Molecular Biology*, 2084, 1–31. https://doi.org/10.1007/978-1-0716-0030-6_1
- [17] Gabelica, V., Shvartsburg, A. A., Afonso, C., Barran, P., Benesch, J. L., Bleiholder, C., Bowers, M. T., Bilbao, A., Bush, M. F., Campbell, J. L., Campuzano, I. D., Causon, T., Clowers, B. H., Creaser, C. S., De Pauw, E., Far,

- J., Fernandez-Lima, F., Fjeldsted, J. C., Giles, K., . . . Wytenbach, T. (2019). Recommendations for reporting ion mobility Mass Spectrometry measurements. *Mass Spectrometry Reviews*, 38(3), 291–320. <https://doi.org/10.1002/mas.21585>
- [18] Wisanpitayakorn, P., Sartyoungkul, S., Kurilung, A., Sirivatanauksorn, Y., Visessanguan, W., Sathirapongsasuti, N., & Khoomrung, S. (2024). Accurate Prediction of Ion Mobility Collision Cross-Section Using Ion's Polarizability and Molecular Mass with Limited Data. *Journal of Chemical Information and Modeling*, 64(5), 1533–1542. <https://doi.org/10.1021/acs.jcim.3c01491>
- [19] May, J. C., Morris, C. B., & McLean, J. A. (2016). Ion Mobility Collision Cross Section Compendium. *Analytical Chemistry*, 89(2), 1032–1044. <https://doi.org/10.1021/acs.analchem.6b04905>
- [20] Feuerstein, M. L., Hernández-Mesa, M., Valadbeigi, Y., Bizec, B. L., Hann, S., Dervilly, G., & Causon, T. (2022b). Critical evaluation of the role of external calibration strategies for IM-MS. *Analytical and Bioanalytical Chemistry*, 414(25), 7483–7493. <https://doi.org/10.1007/s00216-022-04263-5>
- [21] Kartowikromo, K. Y., Olajide, O. E., & Hamid, A. M. (2023). Collision cross section measurement and prediction methods in omics. *Journal of Mass Spectrometry*, 58(9), e4973. <https://doi.org/10.1002/jms.4973>
- [22] David, L., Thakkar, A., Mercado, R., & Engkvist, O. (2020). Molecular representations in AI-driven drug discovery: a review and practical guide. *Journal of Cheminformatics*, 12(1), 56. <https://doi.org/10.1186/s13321-020-00460-5>
- [23] Heller, S. R., McNaught, A., Pletnev, I., Stein, S., & Tchekhovskoi, D. (2015). INCHI, the IUPAC International Chemical Identifier. *Journal of Cheminformatics*, 7(1). <https://doi.org/10.1186/s13321-015-0068-4>
- [24] Prohaska, T., Irrgeher, J., Benefield, J., Böhlke, J. K., Chesson, L. A., Coplen, T. B., Ding, T., Dunn, P. J. H., Gröning, M., Holden, N. E., Meijer, H. a. J., Moossen, H., Possolo, A., Takahashi, Y., Vogl, J., Walczyk, T., Wang, J., Wieser, M. E., Yoneda, S., . . . Meija, J. (2022). Standard atomic weights of the elements 2021 (IUPAC Technical Report). *Pure and Applied Chemistry*, 94(5), 573–600. <https://doi.org/10.1515/pac-2019-0603>
- [25] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). Attention is all you need. *Advances in Neural Information Processing Systems*, 30, 5998–6008. <https://doi.org/10.48550/arXiv.1706.03762>
- [26] Radford, A., Wu, J., Child, R., Luan, D., Amodei, D., & Sutskever, I. (2019). *Language models are unsupervised multitask learners* [Technical report]. OpenAI. https://cdn.openai.com/better-language-models/language_models_are_unsupervised_multitask_learners.pdf
- [27] Ouyang, L., Wu, J., Jiang, X., Almeida, D., Wainwright, C. L., Mishkin, P., Zhang, C., Agarwal, S., Slama, K., Ray, A., Schulman, J., Hilton, J., Kelton, F., Miller, L., Simens, M., Askell, A., Welinder, P., Christiano, P., Leike, J., & Lowe, R. (2022b). Training language models to follow instructions with human feedback. *arXiv (Cornell University)*. <https://doi.org/10.48550/arxiv.2203.02155>
- [28] Hinton, G., Vinyals, O., & Dean, J. (2015). Distilling the knowledge in a neural network. *arXiv (Cornell University)*. <https://doi.org/10.48550/arxiv.1503.02531>
- [29] Micikevicius, P., Narang, S., Alben, J., Diamos, G., Elsen, E., Garcia, D., Ginsburg, B., Houston, M., Kuchaiev, O., Venkatesh, G., & Wu, H. (2017). Mixed Precision training. *arXiv (Cornell University)*. <https://doi.org/10.48550/arxiv.1710.03740>
- [30] Liu, P., Yuan, W., Fu, J., Jiang, Z., Hayashi, H., & Neubig, G. (2022). Pre-train, Prompt, and Predict: A systematic survey of prompting methods in natural language processing. *ACM Computing Surveys*, 55(9), 1–35. <https://doi.org/10.1145/3560815>
- [31] Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W., Rocktäschel, T., Riedel, S., & Kiela, D. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. *Advances in Neural Information Processing Systems*, 33, 9459–9474. <https://doi.org/10.48550/arXiv.2005.11401>
- [32] Wei, J., Wang, X., Schuurmans, D., Bosma, M., Ichter, B., Xia, F., Chi, E., Le, Q., & Zhou, D. (2022, January 28). *Chain-of-Thought prompting elicits reasoning in large language models*. arXiv.org. <https://arxiv.org/abs/2201.11903>
- [33] deepseek-ai/DeepSeek-R1 · Hugging Face. (2025). <https://huggingface.co/deepseek-ai/DeepSeek-R1>
- [34] Holtzman, A., Buys, J., Du, L., Forbes, M., & Choi, Y. (2019). The curious case of neural text degeneration. *arXiv (Cornell University)*. <https://doi.org/10.48550/arxiv.1904.09751>
- [35] Keskar, N. S., McCann, B., Varshney, L. R., Xiong, C., & Socher, R. (2019). CTRL: a Conditional Transformer Language Model for Controllable Generation. *arXiv (Cornell University)*. <https://doi.org/10.48550/arxiv.1909.05858>

- [36] Baker, E. S., Hoang, C., Uritboonthai, W., Heyman, H. M., Pratt, B., MacCoss, M., MacLean, B., Plumb, R., Aisporna, A., & Siuzdak, G. (2023). METLIN-CCS: an ion mobility spectrometry collision cross section database. *Nature Methods*, 20(12), 1836–1837. <https://doi.org/10.1038/s41592-023-02078-5>

8 APPENDIX: Artificial Intelligence Usage

In compliance with the university's regulations on the use of artificial intelligence tools in the Final Degree Project, the following describes how such tools were used throughout the development of this work. The main tools used across all categories described below were Claude (Anthropic) and ChatGPT (OpenAI). No AI tools were used for the generation or modification of images or diagrams. As the author, I take full responsibility for the entire content of this TFG and can defend them in full, having made all final wording, design, implementation, and analysis decisions throughout the project.

8.1 Writing, drafting and style review

Both tools were used as editorial support during the writing of some sections. This consisted of proposing alternative phrasings for specific passages where the original wording was imprecise or redundant, reviewing the internal coherence of paragraphs drafted, identifying unnecessary repetitions, and suggesting vocabulary alternatives. No spell-checkers other than those integrated in the word processor were used.

8.2 Code generation and debugging

Claude was specifically used to assist in debugging some errors in the Python scripts, particularly in the response parsing logic, the classification of predictions, and the integration of the pipeline with the Flask application. I was responsible for the system architecture design and all implementation criteria, while Claude served as a reference resource for resolving specific problems. Additionally, the HTML interface used in the web application was generated by Claude and then personalized, given that the user interface was not a primary focus of this project but rather a convenience for future use.

8.3 Other uses

In the early stages of the project, ChatGPT was used to clarify specific concepts from the articles consulted, particularly in cheminformatics and language models, to better understand the content and resolve additional doubts. This understanding informed the initial drafting of the theoretical framework (Chapter 2).